Ref. Ares(2024)6901285 - 30/09/2024

Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project funded by the European Innovation Council and SMEs Executive Agency

D5.2 Extensive awareness models for safe and resilient UTM

Work Package: WP5

Authors: Sybert Stroeve, Mirco Kroon, Patrick Jonk, Arash Bahari Kordabad, Andrea Da Col, Arabinda Ghosh, Sadegh Soudjani

Project Title: Symbolic Logic Framework for Situational Awareness in Mixed Autonomy

Project Acronym: SymAware

Grant Agreement number: 101070802

Project Start Date: 1 October 2022

Duration: 36 months

Project Coordinator: Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Project Members:

Max-Planck-Gesellschaft Zur Forderung Der Wissenschaften EV (MPI-SWS)

Technische Universiteit Eindhoven (TUE)

Kungliga Tekniska Hoegskolan (KTH)

Uppsala Universitet (UU)

Stichting Koninklijk Nederlands Lucht - En Ruimtevaartcentrum (NLR)

Siemens Industry Software Netherlands BV (SISW)

Document Identifier: D5.2

Issue: 1.0

Issue Date: 30 September 2024

Prepared by: NLR, MPI-SWS **Approved:** MPI-SWS

Dissemination Status: Public

History Chart:

Issue	Date (DD/MM/YYYY)	Updated Sections	Cause of Change	Implemented by
0.1	01.08.2023	All sections	New Document	Sybert Stroeve
0.2	21.03.2024	Sections 2, 4		Sybert Stroeve
0.3	28.05.2024	Section 5		Sybert Stroeve, Patrick Jonk, Mirco Kroon
0.4	16.09.2024	Sections 3.1, 3.2		Arash Bahari Kordabad, Andrea Da Col, Arabinda Ghosh, Sadegh Soudjani, Sybert Stroeve
0.5	17.09.2024	Section 3.3		Sybert Stroeve, Mirco Kroon
1.0	30.09.2024		Final version following review	Sybert Stroeve

All rights reserved

The document is proprietary of the SymAware consortium members listed on the front page of this document. The document is supplied on the express understanding that it is to be treated as confidential and may not be used or disclosed to others in whole or in part for any purpose except as expressly authorized in terms of Grant Agreement number 101070802. The SymAware consortium makes no warranty for the information contained in this document; neither does it assume any legal liability or responsibility for the accuracy, completeness or usefulness of this information. Company or product names mentioned in this document may be trademarks or registered trademarks of their respective companies.

Contents

1 Introduction 8
1.1 SymAware: High-level description of the project 8
1.2 Purpose and contents of this report 9
2 Extending detect and avoid systems 11
2.1 Standards 11
2.1.1 DAA Well Clear definition 11
2.1.2 ACAS X 12
2.1.3 DAIDALUS 13
2.1.4 Limitations 14
2.2 Review of literature in support of extended DAA systems 14
2.3 Towards intent and risk aware DAA systems 16
3 Intent-aware DAA systems 19
3.1 Dynamic programming 19
3.1.1 Markov Decision Process 19
3.1.2 Dynamic programming in existing ACAS and DAA systems 19
3.1.3 MDP for intent-based detect and avoid 21
3.2 Robust Model Predictive Control 23
3.2.1 Model Predictive Control 23
3.2.2 Robust scenario-tree MPC 24
3.2.3 Intent-aware scenario-tree MPC-based DAA in the horizontal plane 25
3.3 A* path planning-based conflict resolution 28
3.3.1 Conflict risk cost function 28
3.3.2 A* path planning per aircraft 29
3.3.3 Path planning for a set of aircraft 31
4 Models for evaluation by sets of encounters 33

4.1 Detect and avoid system 33
4.2 Response delay and remote pilot 33
4.3 Closest point of approach in encounters with straight trajectories 34
5 Methods for risk assessment in a simulation environment 37
5.1 Scope 37
5.1.1 Concept of operation 37
5.1.2 Variability and nonnominal modes 37
5.2 Agent-based modelling 38
5.2.1 Overview of the agent-based model 38
5.2.2 Modelling of nominal and off-nominal conditions 41
5.3 Rare event Monte Carlo simulation 42
5.3.1 IPS MC simulation 42
5.3.2 Risk decomposition for global failure conditions 44
6 Bibliography 45

List of Figures

Figure 1. The novel cognititive architecture developed in SymAware 9

Figure 2. The main (technical) work packages of SymAware 9

Figure 3. Comparison of a state-based DAA system with a DAA system based on state/intent/risk information 17

Figure 4. Example of a conflicting aircraft pair, where the conflict is resolved by providing auxiliary waypoints AB1 and AB2 to aircraft 1 and 2. The left figure shows the flights in the horizontal xy-plane, while the right figure shows altitude versus time 18

Figure 5. Aircraft encounter geometry with intent information for horizontal collision avoidance system ... 22

Figure 6. The evolution of the system represented as a scenario-tree 24

Figure 7. An LSR Dubin's path from an initial to a final state. 26

Figure 8. Nodes and edges for path finding for horizontal manoeuvring, source [34] 30

Figure 9. High-level overview of the agent-based model, from [54] 39

Figure 10. Schematic diagram of Working and Failing modes of local and global system characteristics A and B influencing system performance of aircraft i. D1 to D8 are transitions with exponentially distributed delays. Source: [54] 42

Figure 11. Illustration of IPS MC simulation for estimation of rare miss distances and collisions. Source: [54] 43

List of Tables

Table 1. Parameters in the definition of DAA Well Clear for various types of operations of large drones [2] 12

Table 2. Normal variability and failure modes in the scope of the models..... 38

List of Abbreviations

ACAS	Airborne Collision Avoidance System
ADS-B	Automatic Dependent Surveillance - Broadcast
ACAS	Airborne Collision Avoidance System
AGL	Above Ground Level
AI	Artificial Intelligence
ATC	Air Traffic Control
ATM	Air Traffic Management
BVLOS	Beyond Visual Line Of Sight
CA	Collision Avoidance
ConOps	Concept of Operations
DAA	Detect And Avoid
DAIDALUS	Detect and Avoid Alerting Logic for Unmanned Systems
EIC	European Innovation Council
FAA	Federal Aviation Administration
FMS	Flight Management System
MAS	Multi Agent System
MDP	Markov Decision Process
MOPS	Minimum Operational Performance Standards
NA	Not Applicable
NASA	National Aeronautics and Space Administration
NMAC	Near Mid-Air Collision
NOTAM	Notice to Airmen
OSED	Operational Services and Environment Definition
POMDP	Partially Observable Markov Decision Process
RA	Resolution Advisory
RP	Remote Pilot
RPIC	Remote Pilot In Command
RWC	Remain Well Clear
SA	Situational Awareness
SESAR	Single European Sky Air Traffic Management Research
STM	Surveillance and Tracking Module

sNMAC	sUAS Near Mid-Air Collision
sUAS	small Unmanned Aircraft System
TA	Traffic Advisory
TCAS	Traffic alert and Collision Avoidance System
TRM	Threat Resolution Module
UA	Unmanned Aircraft
UAM	Urban Air Mobility
UAS	Unmanned Aircraft System
USS	UAS Service Supplier
UTM	UAS Traffic Management
VLL	Very-Low Level

1 Introduction

1.1 SymAware: High-level description of the project

SymAware addresses the fundamental need for a new conceptual framework for awareness in multi-agent systems that is compatible with the internal models and specifications of robotic agents and that enables safe simultaneous operation of collaborating autonomous agents and humans. The goal of SymAware is to provide a comprehensive framework for situational awareness to support sustainable autonomy via agents that actively perceive risks and collaborate with other robots and humans to improve their awareness and understanding while fulfilling complex and dynamically changing tasks. The SymAware framework will use compositional logic, symbolic computations, formal reasoning, and uncertainty quantification to characterise and support situational awareness of MAS in its various dimensions, sustaining awareness by learning in social contexts, quantifying risks based on limited knowledge, and formulating risk-aware negotiation of task distributions. These goals will be achieved in SymAware through the following technical objectives:

1. Logical characterization of awareness using symbolic methods;
2. Quantifying the symbolic reasoning for awareness with spatial and temporal ingredients for decision making;
3. Risk awareness via quantified knowledge;
4. Quantifying and communicating knowledge awareness;
5. Demonstrating awareness engineering in aviation and automotive use cases; and
6. Identifying requirements for ethical and trustworthy awareness in human-agent interaction.

The objectives of SymAware address the “Awareness Inside” Challenge of the European Innovation Council (EIC) by extending and formalizing human-based models of situational awareness and by providing a novel conceptual situational awareness framework for MASs that encompasses logical characterisation and integrative formal reasoning of interdependent awareness dimensions including knowledge, spatiotemporal, risk and social dimensions. This will support transitioning to safe mixed operation of autonomous agents and humans.

Figure 1 shows a schematic view of the SymAware new cognitive awareness architecture. Together the objectives of this proposal will achieve the depicted situational awareness for multi-agent settings in which the situational awareness is related to the different awareness dimensions and to contemporary methods for probabilistic modelling, perception, estimation, and (data-driven) computation.

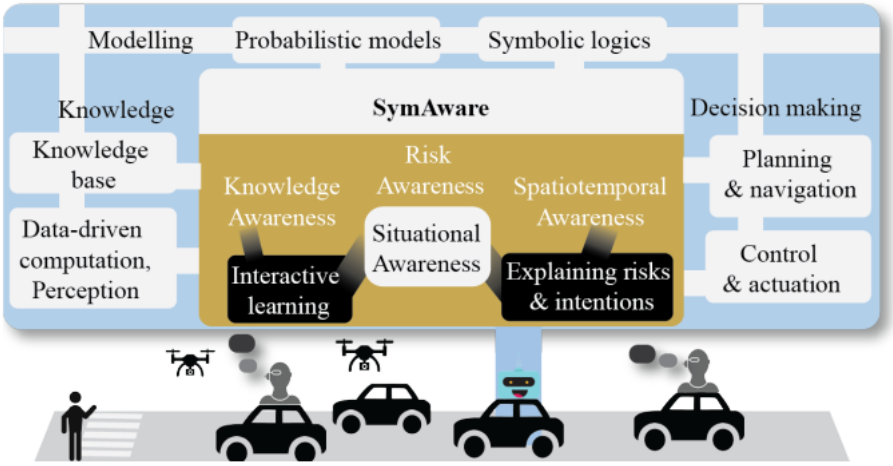

Figure 1. The novel cognititive architecture developed in SymAware

Figure 2 shows the main technical work packages of SymAware. The comprehensive framework of situational awareness and its logical characterisation (Objective 1) will be developed in WP1. The spatiotemporal awareness of Objective 2 will be studied in WP2. Risk and knowledge awareness (Objectives 3 and 4) will be studied in WP3 and WP4, respectively. The automotive and aviation use cases (Objective 5) will be developed in WP5. Finally, the human-robot interaction and the ethical requirements (Objective 6) will be studied in WP6.

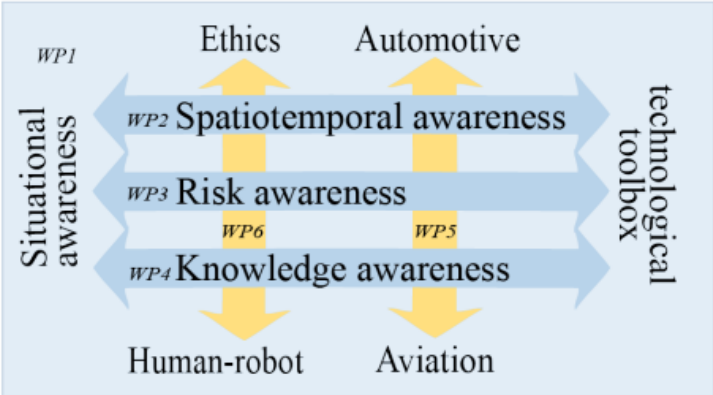

Figure 2. The main (technical) work packages of SymAware

1.2 Purpose and contents of this report

This report is the second deliverable of WP5 on the automotive and aviation use cases. Its purpose is to develop methods that extend the situational awareness in unmanned aircraft system traffic management (UTM), so as to improve detect-and-avoid (DAA) systems that support safe separation between unmanned aircraft systems (UAS). This development is based on initial work on situational and risk awareness in a UTM operational concept presented in D5.1 [1]. The contents of this report provide the basis for D5.3, which will present results on the evaluation of the safety impact and robustness of the developed methods.

This report is structured in the following way:

- Section 1 describes the SymAware project and the purpose of the current report.
- Section 2 provides an overview of standards for DAA systems applied in UAS operations, a review of selected literature in support of extending DAA systems, and the concept of intent and risk aware DAA systems that are developed in SymAware.
- Section 3 presents a number of methods to achieve intent-aware DAA systems, including a dynamic programming approach with intent information, a robust model predictive control approach that applies scenario trees, and a path planning approach that includes intent and risk information.
- Section 4 describes models of encounter scenarios in that will be used to compare DAA approaches in the aviation use case.
- Section 5 describes a risk assessment approach for evaluation of DAA systems in the scope of an operational concept, variability factors in operations, and non-nominal operating modes.

2 Extending detect and avoid systems

This chapter provides an overview of standards for detect and avoid systems applied in UAS operations (Section 2.1), a review of selected literature in support of extending DAA systems (Section 2.2), and the concept of intent and risk aware DAA systems that are developed in SymAware (Section 2.3).

2.1 Standards

The purpose of a detect and avoid (DAA) system for UAS operations is to support maintaining sufficient separation with other aircraft and potentially ground objects so as to minimize the probability of a collision below sufficient target levels of safety. This section describes some standards which have been published by aviation industry standardization organizations RTCA and/or EUROCAE.

2.1.1 DAA Well Clear definition

In Appendix C.5 of [2] a DAA Well Clear (DWC) definition is provided for an aircraft pair (i, j) with positions (x_i, y_i, z_i) and (x_j, y_j, z_j), and horizontal speeds $(\dot{x}_i, \dot{y}_i)$ and $(\dot{x}_j, \dot{y}_j)$. The horizontal range between the aircraft is

$$r = \sqrt{r_x^2 + r_y^2} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \tag{1}$$

where $r_x = x_i - x_j$ and $r_y = y_i - y_j$ are the distances between the aircraft in the x and y dimensions. The vertical spacing between the aircraft is

$$h = z_i - z_j \tag{2}$$

The horizontal range rate between the aircraft is

$$\dot{r} = \frac{r_x \dot{r}_x + r_y \dot{r}_y}{r} \tag{3}$$

where $\dot{r}_x = \dot{x}_i - \dot{x}_j$ and $\dot{r}_y = \dot{y}_i - \dot{y}_j$ are the relative speeds in the x and y dimensions. The range rate is negative for closing geometries. The closest point of approach (CPA) is attained when the range rate is zero, $\dot{r} = 0$. For closing geometries and assuming constant speed vectors the time of CPA is now projected to be

$$\hat{t}_{CPA} = -\frac{r_x \dot{r}_x + r_y \dot{r}_y}{\dot{r}_x^2 + \dot{r}_y^2} \tag{4}$$

The projected horizontal miss distance is

$$\hat{r}_{CPA} = \sqrt{(r_x + \dot{r}_x \hat{t}_{CPA})^2 + (r_y + \dot{r}_y \hat{t}_{CPA})^2} \tag{5}$$

For closing geometries, a so-called modified tau τ_{mod} is defined as

$$\tau_{\text{mod}} = \begin{cases} -\dfrac{r^2 - r_{\text{mod}}^2}{r\dot{r}} & \text{if } r > r_{\text{mod}} \\ 0 & \text{if } r \le r_{\text{mod}} \end{cases} \tag{6}$$

where r_{mod} is a distance modification term, which describes the radius of a protection disk around an ownship, such that any intruder within this distance is an urgent threat. It can be recognized that for large ranges $r \gg r_{\text{mod}}$, the modified tau is about equal to the range over the closure range rate $\tau_{\text{mod}} \approx -r / \dot{r}$.

Using above elements a loss of DAA well clear can be defined by boolean B_{LDWC}

$$B_{LDWC} \equiv (\tau_{\text{mod}} \le \tau_{\text{mod}}^*) \wedge (\hat{r}_{CPA} \le r_{CPA}^*) \wedge (|h| \le h^*) \tag{7}$$

where τ_{mod}^*, r_{CPA}^* and h^* are threshold values for the modified tau, projected horizontal miss distance, and vertical separation, respectively. Values for these threshold values depend on the type of operation, as shown in Table 1.

Table 1. Parameters in the definition of DAA Well Clear for various types of operations of large drones [2]

Parameter	**Description**	**Operation**		
		En route cooperative	**En route non-cooperative**	**Terminal**
τ_{mod}^*	Modified tau threshold	35 s	0 s	0 s
r_{mod}	Distance modification of modified tau	4000 ft	2200 ft	1500 ft
r_{CPA}^*	Horizontal miss distance threshold	4000 ft	2200 ft	1500 ft
h^*	Vertical separation threshold	450 ft	450 ft	450 ft

2.1.2 ACAS X

The ACAS X programme is FAA sponsored R&D towards a more advanced ACAS [3, 4]. The basis of ACAS X systems are probabilistic estimations of ownship and othership states, which are combined with optimized decision logic tables to achieve advised actions. The optimized decision logic tables have been determined off-line using a partially observable Markov decision process (POMDP) model and dynamic programming [5, 6]. This optimization process uses (1) probabilistic dynamic models for pilot response

and aircraft movements, (2) a reward function with an extensive set of cost components for close proximities and types of advisories, and (3) dynamic programming to maximize the reward function, leading to state-action values that are stored in large look-up tables. Given a perfect model of the environment as a finite Markov decision process (MDP), dynamic programming is guaranteed to lead to an optimal policy for the finite states in the environment. Dynamic programming can be seen as off-line reinforcement learning for completely known environments [7].

The following variants of ACAS X are relevant for UAS and/or UAM:

- ACAS Xu is designed for large UAS and it provides Remain Well Clear (RWC) functionality in addition to a Collision Avoidance functionality [8, 9]. It is able to use non-cooperative surveillance inputs, such as air-to-air radar in its STM. It makes use of alerting and guidance in the horizontal plane as well as the vertical plane, and these are provided independently. The advisories specified by ACAS Xu include alerts and guidance to remain well clear of other aircraft by avoiding particular ranges of headings and/or vertical speeds, as well as resolution advisories specifying to adjust the aircraft towards particular ranges of headings and/or vertical speeds. Resolution advisories are coordinated between ACASs, while RWC advisories are not coordinated.
- ACAS sXu is designed for smaller UAS with a wingspan of 25 feet or less and height of 12 feet or less, which are expected to operate mostly below 1200 ft AGL [10, 11]. The STM uses surveillance data to maintain tracks of nearby aircraft as well as terrain/obstacles. The TRM uses decoupled vertical and horizontal components. All sXu guidance is directive, allowing for auto-response. In particular, The advisories specified by ACAS sXu are resolution advisories, specifying to adjust the aircraft towards particular ranges of headings and/or vertical speeds. The resolution advisories are coordinated between ACASs of involved aircraft. The nominal response delay for auto-response to RAs is less than 1 s, independent from the type of RA. The protection volume for sUAS intruders is based on 50 ft horizontal and ±15 ft vertical volume, which is called sUAS Near Mid-Air Collision (sNMAC) [12]. Towards manned and large UAS intruders the sXu logic applies a RWC volume of 2000 ft horizontally and ±250 ft vertically. Selection of a protection volume based on intruder type is called dynamic scaling.
- ACAS Xr is being developed for rotorcraft operations, including air taxis.

2.1.3 DAIDALUS

DAIDALUS (Detect and Avoid Alerting Logic for Unmanned Systems) is a DAA reference system of the RTCA MOPS for DAA [2] developed by NASA. These MOPS regard DAA systems that assist remote pilots of larger UAS; they do not apply to small UAS operating in low-level environments. Two aircraft are considered to be well clear of each other, if appropriate distance and time variables determined by the relative aircraft states remain outside a set of predefined threshold values. The alerting logic of DAIDALUS is rule-based, in contrast to the use of optimized look-up tables in ACAS Xu. The alerting logic of DAIDALUS is explained in[13].

The advisories specified by DAIDALUS includes alerts and guidance to remain well clear of other aircraft by avoiding particular ranges of headings, altitudes, or speeds, as well as alerts and guidance for recovery of the well clear condition by advising to adjust the aircraft towards particular ranges of headings, altitudes, or speeds. There is no coordination between DAIDALUS at different aircraft.

2.1.4 Limitations

It is explicitly indicated in the UTM ConOps of FAA [14] that shared situational awareness and shared intent are important means for assuring sufficient separation between aircraft. However, existing and developing DAA systems are based only on current absolute or relative state estimates (position, velocity) of involved aircraft; this applies to ACAS Xu, ACAS sXu, ACAS Xr, DAIDALUS. This means that future (intended) positions, as considered in a flight plan, are not used as means to remain well clear or for collision avoidance. Also RWC advisories are not shared or coordinated between DAA systems, which restricts attaining shared situational awareness about these manoeuvres. These limitations may have repercussions on the effectiveness of assuring separation and on the efficiency of the advisories with respect to flight time and energy consumption. For instance, it was shown that in particular encounters of pairs of remotely piloted drones that are both equipped with ACAS Xu, instability may exist leading to deadlock conditions wherein the drones do not manage to pass each other [15]. Furthermore, current designs of prevalent DAA systems are mostly focused on conflicts with other aircraft and may include some ground obstacles. They do not take explicitly into account other risks, such as collisions with humans, birds or an empty battery. So the SymAware framework for shared situational awareness and risk awareness are in line with general concepts of operations for UTM and they may help to overcome limitations of current DAA systems.

2.2 Review of literature in support of extended DAA systems

Most DAA guidance algorithms are of a tactical nature and do not directly take into account potential challenges or cost of returning the ownship to its flight plan path afterwards. DAA manoeuvre planning that extends beyond RWC advisories by considering route recapture is studied in [16]. It is shown that optimal horizontal trajectories and the auto-resolver of [17] use turns into rather than turns away from the othership more frequently than DAIDALUS. The rule-based system [17] was proposed for automated conflict resolution for air traffic control, which uses auxiliary waypoints; the same concept is used for avoiding bad weather zones in [18]. Its first main component is a Resolution Aircraft and Manoeuvre Selector (RAMS). This selects a single aircraft for making a manoeuvre to resolve a conflict as well as the type of manoeuvre (horizontal, vertical, speed). The second main component is the Resolution Manoeuvre Generator (RMG). This determines the altitude step, timing, auxiliary waypoints and/or speeds for the chosen aircraft and manoeuvre type. So this system resolves the conflict by manoevring a single aircraft in a single dimension.

Reward function optimisation of a deep reinforcement learning for a collision avoidance system has been studied in [19]. In this study a deep neural network was directly optimized using reinforcement learning. This allows to have a high-dimensional input state space and to combine the optimization of advisories in the horizontal and vertical planes. The achieved performance was however poorer than that of ACAS Xu.

Several applications of reinforcement learning for UAS operations were studied by Ribeiro et al. [20-24]. They mostly used state-based information for conflict detection and resolution, including relative headings, relative altitudes, and distances to neighboring drones. In [22] RL was used to control heading, speed, and vertical speed to avoid loss of separation. In a comparison with the modified voltage potential (MVP) geometric resolution approach the performance of the RL-based control in avoiding losses of minimum separation was shown to be poorer. In [21] RL learning was used in an hybrid approach in combination

with MVP, where it was trained to determine the look-ahead time and the dimension of the control action based on the conflict geometry. The results showed that the total number of losses of minimum separation could be reduced by the hybrid approach in comparison with MVP only. In [24] also intent information was included in an urban environment use case that applied speed-only resolution. Here a conservative approach was followed were conflicts were detected either by the state or the intent information, such that the number of conflicts increased and the number of possible solutions decreased.

A safety evaluation of an advanced self-separation concept for trajectory-based operations of manned aircraft is presented in [25-27]. In this concept aircraft share intended four-dimensional trajectories with other aircraft and they have an airborne separation assurance systems (ASAS) in each aircraft. Main ASAS components are short-term and medium-term conflict detection and resolution systems. The ASAS functionality is intent-based, meaning that the intent information from own and other aircraft is used, if the intent information is considered to be reliable (following conformance monitoring); otherwise state information is used. The medium- and short-term conflict resolution is achieved in the horizontal plane by heading changes based on velocity obstacles [28]. The results show that the concept can safely accommodate high en-route traffic demands.

An analytical closed mathematical formulation of the velocity obstacles is presented in [29]. It allows to include intent information and to account for trajectory prediction errors. The velocity obstacles approach can be represented in a state space diagram. In [30] such state space diagrams were used to obtain input features for conflict resolution obtained by deep reinforcement learning.

An approach for Markov decision process-based distributed conflict resolution for drone air traffic management is presented in [31]. It extends methods as applied for the development of ACAS Xu to the resolution and coordination of multi-agent conflicts for drones flying at low altitudes.

A distributed cooperative onboard conflict resolution approach for UASs is presented in [32]. A safe separation constraint is based on a geometric conflict resolution method. This constraint is used in a distributed constraint optimization method, which applies a stochastic gradient descent algorithm. In [33] the same research group used a space mapping iterative search method.

In [34] a method is proposed which is based on a probabilistic conflict risk map using the predicted probability of conflict with the intention information of the intruders. The risk-map-based method allows the path planning algorithm to simultaneously account for uncertainties. The A* path planning algorithm is used to find the cost-minimized trajectory for a single aircraft by considering all other aircraft as intruders. Here the action space is a constant heading change over a particular time-step, thus representing a particular turn rate. A search heuristic method is used to iterate the A* algorithm for all aircraft to optimize the trajectory planning. Results are shown for manoeuvring in the horizontal plane and for disturbances like wind. It is explained how 3D conflict resolution can be achieved by combination with a path planning approach for vertical conflict resolution [35]. A meta-controller is used which first tries to solve the conflict horizontally and at a later stage vertically. Results of [35] show that collision avoidance for multiple aircraft scenarios can be attained by vertical manoeuvring.

A survey of path planning algorithms is provided in [36]. In [37] hybrid-state A* path planning for autonomous vehicles was developed, which accounts for the continuous dynamics of the vehicle. A review of path planning for autonomous drones is provided in [38]. It provides an overview of (recent) papers for path planning properties like 2D or 3D planning, the inclusion of energy constraints, trajectory constraints,

types of drones, static and/or dynamic obstacles, obstacle density, and the type of algorithm. Path planning strategies for UAVS in 3D environments using A* and Theta* are presented in [39]. An adaptive path replanning approach for drones in urban environments is presented in [40]. Here dynamic path replanning is performed to avoid no-fly zones, cooperative drones, and non-cooperative drones. Path planning for collision avoidance of UASs with non-cooperative air traffic using a closed-loop rapidly- exploring random tree algorithm has been developed in [41].

2.3 Towards intent and risk aware DAA systems

In support of the aviation use case in SymAware intent and risk aware DAA systems will be developed, which are aimed to extend the DAA performance beyond what can be achieved by existing state-based DAA systems.

A variety of input variables can be available for the new DAA systems, including:

- 3D position and speed estimates of the ownship (ownship state);
- 3D position and speed estimates of nearby other aircraft (intruder state);
- 3D position and speed at one or several next waypoints in the flight planning of the ownship (ownship intent);
- 3D position and speed at one or several next waypoints in the flight planning of nearby other aircraft (intruder intent);
- battery states of ownship and other aircraft in relation to battery needed for remaining flight.

The following output variables of the new DAA systems are considered:

- advised headings and vertical speeds;
- advised additional intermediate waypoint (3D position) of the ownship, which is achieved after coordination with an intruder.

A comparison of a (current) state-based DAA system with a (new) DAA system that also employs intent and risk information, and that provides changes in the intent (new waypoints) is shown in Figure 3.

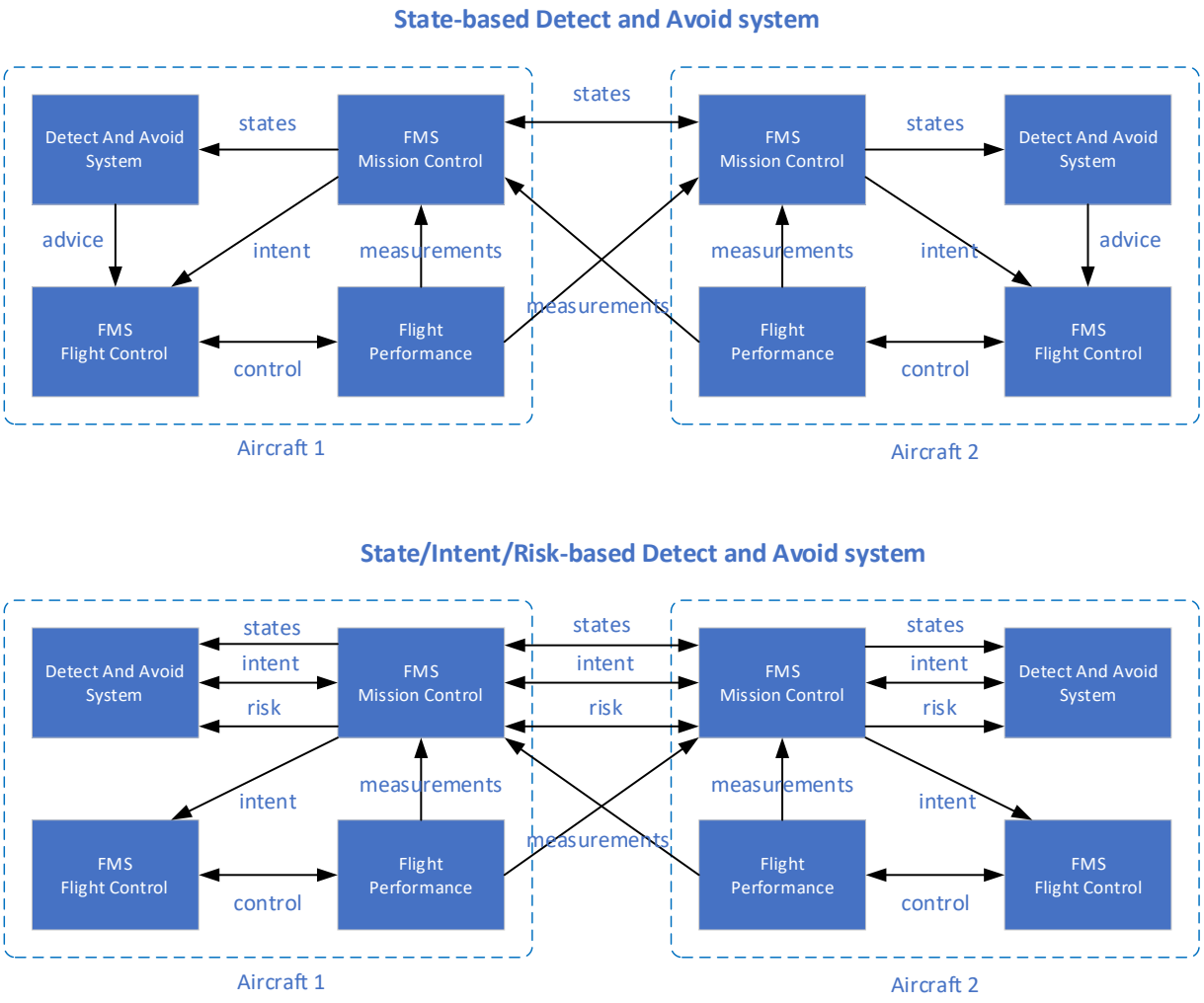

Figure 3. Comparison of a state-based DAA system with a DAA system based on state/intent/risk information

As an example Figure 4 shows a conflict between two aircraft 1 and 2, which have planned to fly from waypoints A1 to B1 and C1, and from A2 to B2 and C2, respectively. The DAA systems provide auxiliary intermediate waypoints AB1 and AB2 to aircraft 1 and 2, respectively. In general such waypoints may imply that the aircraft have to change heading and/or altitude. In this example the intermediate waypoints imply manoeuvres in the horizontal plane only. After attaining the auxiliary waypoint each aircraft changes its trajectory to its waypoint B1/B2. There exists coordination between the aircraft, which implies sharing of the auxiliary waypoints and some kind of negotiation or local optimization for the conflict between the involved aircraft. This negotiation / optimization should be focused on attaining advisories that minimize the risks of both aircraft.

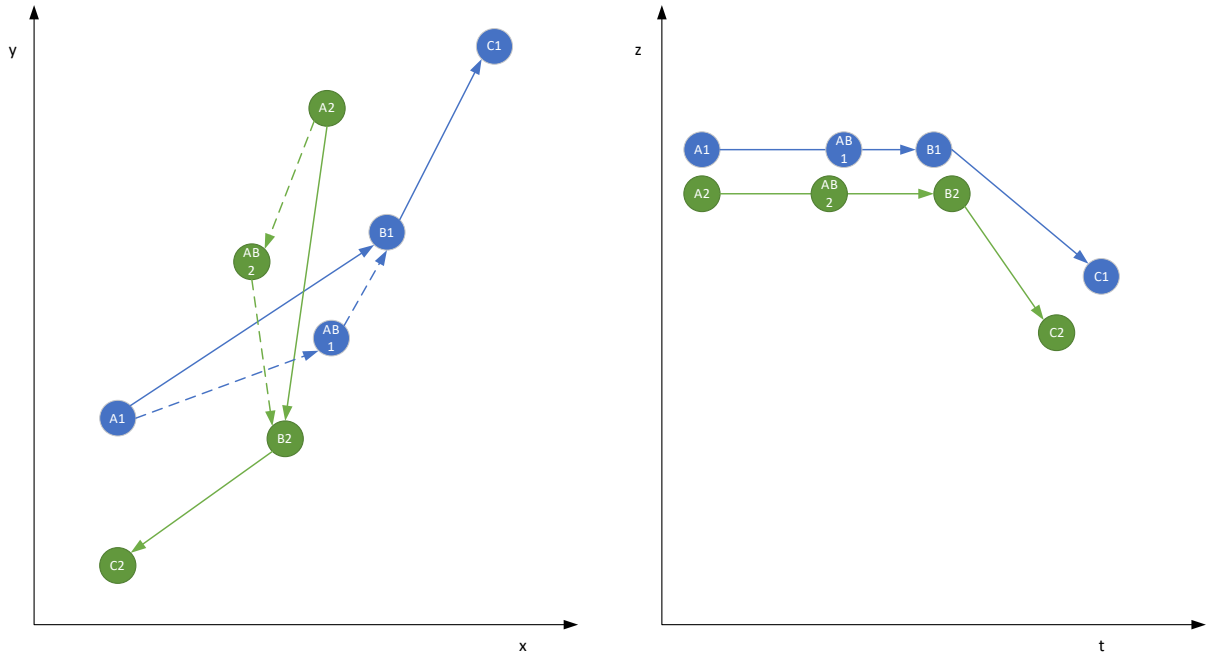

Figure 4. Example of a conflicting aircraft pair, where the conflict is resolved by providing auxiliary waypoints AB1 and AB2 to aircraft 1 and 2. The left figure shows the flights in the horizontal xy-plane, while the right figure shows altitude versus time

3 Intent-aware DAA systems

This chapter presents a number of methods to achieve intent-aware detect-and-avoid systems. Section 3.1 describes a dynamic programming approach that has included intent information. Section 3.2 describes a robust model predictive control approach that applies scenario trees. Section 3.3 describes a path planning approach that includes intent, separation and risk information.

3.1 Dynamic programming

3.1.1 Markov Decision Process

A Markov Decision Process (MDP) is a 4-tuple $(\mathcal{S}, \mathcal{A}, T, L)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $T : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R} \in [0,1]$ is the transition probability function and $L : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the immediate cost function. In every state $\mathbf{s} \in \mathcal{S}$, the system can execute an action $\mathbf{a} \in \mathcal{A}$ and reach a new state $\mathbf{s}^+ \in \mathcal{S}$ with probability $T(\mathbf{s}^+ | \mathbf{s}, \mathbf{a})$. In addition, the system gets an immediate cost $L(\mathbf{s}, \mathbf{a})$ during this transition. In control theory and for the continuous state spaces, the transition $T(\mathbf{s}^+ | \mathbf{s}, \mathbf{a})$ is equivalently described using a (possibly nonlinear) function as follows:

$$\mathbf{s}^+ = \mathbf{F}(\mathbf{s}, \mathbf{a}, \mathbf{w}), \quad \mathbf{w} \sim W \tag{8}$$

here $\mathbf{w}$ is a random disturbance drawn from a time-invariant distribution W.

A policy $\boldsymbol{\pi} : \mathcal{S} \to \mathcal{A}$ maps states to actions and describes how to choose an action in each state. The goal is to find an optimal policy $\boldsymbol{\pi}^\star(\mathbf{s})$ that minimizes the following cumulative cost function:

$$V^\star(\mathbf{s}) := \min_{\boldsymbol{\pi}} \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k L(\mathbf{s}_k, \boldsymbol{\pi}(\mathbf{s}_k)) \,\middle|\, \mathbf{s}_0 = \mathbf{s} \right], \tag{9}$$

where $V^\star$ is the optimal value function, $\gamma \in (0,1)$ is the discount factor, and $\mathbf{s}_{k+1} \sim T(\cdot \mid \mathbf{s}_k, \boldsymbol{\pi}(\mathbf{s}_k))$.

Dynamic programming (DP) is a powerful technique that is used to solve MDPs by breaking the problem down into simpler subproblems and solving them recursively. Value iteration and policy iteration are two important algorithms in DP that are used to find optimal policies by iteratively improving value functions or policies based on the Bellman equations. One major drawback of DP is the high memory and time requirements, as it typically involves storing solutions to all subproblems, which can lead to exponential space and time complexity. This makes DP computationally infeasible for very large problems or for problems with a high-dimensional state space.

3.1.2 Dynamic programming in existing ACAS and DAA systems

Dynamic programming for optimization of an MDP was used for the development of standards for the airborne collision avoidance system ACAS Xa and for the detect-and-avoid (DAA) system ACAS Xu.

ACAS Xa [42, 43] provides vertical resolution advisories to pilots of (large) manned aircraft, which are within last-minute advisories to avoid a collision with another aircraft. The Threat Resolution Module of

ACAS Xa uses optimized decision logic stored in look-up tables to determine its advisories given probabilistic estimates of ownship and intruder states. The optimized decision logic tables with state-action values, which have been determined off-line using a partially observable MDP model (POMDP) and DP [5, 6]. For the provision of advisories ACAS Xa extends these precomputed actions with coordination rules for complementary advisories, and with online costs for required system performance, e.g., low altitude inhibitions of descend RAs and RA transition penalties. In the application of DP for the development of the ACAS Xa decision logic there exist various sources of uncertainty [44]. Applied models of aircraft movements and pilot response include epistemic and aleatory uncertainty, and there is no finite state space, but aircraft moving in a time-space continuum, which is approximated using (large numbers of) discretized states. Furthermore, it is not known beforehand what a suitable reward function is. The ACAS Xa development has involved an iterative process in which the operational and safety impact of attained ACAS performance has been evaluated in simulations for particular encounter sets, leading to the adaptation of the reward function [6, 45] and ACAS Xa algorithms. For instance, while early in the development (negative) rewards were assigned only to near collisions and to issuing initial, strengthening and reversal RAs, later in the development considerably more reward components were found to be needed to achieve acceptable performance, where the reward components may depend on particular vertical separations and closure rates (see e.g., Table 10-2 in [6]). As such, the achieved alerting logic in ACAS Xa depends on the POMDP model, the discretization grid, the reward components, the encounters, and settings used in simulations of the dynamic programming optimized logic, the judgement of the simulated performance, and the adaptations of the reward function in the iterations.

In a comparison of the performance of ACAS Xa with the TCAS II system, which is a current rule-based system used in commercial air transport, it was shown that although ACAS Xa can be more effective for collision avoidance in some types of encounters, the impact of the probability of pilot response to the advisories is much larger than performance differences between TCAS II and ACAS Xa [44]. Also it was found that for some types of encounters ACAS Xa was less robust for sensor errors than TCAS II. Possibly sensor errors neglected in the optimization process of ACAS Xa have led to more specific and less robust performance in its alerting logic, while TCAS II uses generic rules.

ACAS Xu is a recent DAA system that incorporates both remain well clear (RWC) and collision avoidance (CA) functions in support of remotely piloted aircraft systems (RPAS) [8, 46]. The RWC function supports detection and analysis of potential conflicting traffic and provides flight path guidance to the remote pilot (RP) to prevent the conflict developing into a collision hazard. The CA function provides last-resort resolution advisories (RAs) to the RP to avoid physical contact between the aircraft. Key differences with ACAS Xa are that (1) ACAS Xu provides both RWC guidance and RAs while ACAS Xa only provides RAs, (2) ACAS Xu provides its guidance and advisories both horizontally and vertically while ACAS Xa only provides vertical advisories, and (3) ACAS Xu provides its output to a remote pilot rather than to an onboard pilot, leading to larger response delays. The threat resolution module (TRM) of ACAS Xu is based on two independent POMDP models for advisories in the vertical plane and the horizontal plane. The optimization for the vertical and horizontal dimensions is separated, since the combined problem was considered intractable to solve due to its large (discretized) state space. The objective function in the optimization specifies costs for ACAS actions and outcomes in simulated encounters. The POMDP is solved through value iteration to calculate a Q-function representing the value gained for taking an action in the current state. In ACAS Xu the Q-functions are represented as lookup tables with a total size of 5 GB. For the provision of RAs, ACAS Xu extends these precomputed actions with coordination rules for

complementary advisories (assuring that they are in opposite directions), and with online costs for required system performance, e.g. low-altitude inhibits of descend RAs, altitude dependent logic sensitivity, and RA transition penalties. The RWC guidance provided by ACAS Xu is based on a rollout approach [47], which uses the CA POMDP-based cost tables to infer an increase in collision risk in relation to DAA alert timing requirements. The RWC guidance does not use coordination between nearby aircraft.

In [48, 49] the results of deterministic and Monte Carlo simulations are provided for a set of encounters between pairs of RPAS both having ACAS Xu for various configurations of the RP model, and for cases with and without sensor errors. The results show that there can be large differences between the results of deterministic and Monte Carlo simulations, indicating that limited sensor errors can have a large impact on the nonlinear system dynamics. Furthermore it is shown that livelock conditions can exist where the RPAS show oscillatory behaviour and do not manage to effectively pass each other, dependent on the encounter geometry and RP model settings. This type of livelock performance indicates a limitation in the scope of current DAA systems [2, 8]. They only provide guidance on how to avoid other traffic, but they do not provide guidance on how to regain the route to the desired destination without inducing new conflicts. For returning to mission the DAA system would need to know the planned route (intent) of the ownship. Preferably also the intent of the intruder would be available, such that a coordinated advice can be provided to remain well clear enduringly. The identified limitation of the state-based ACAS Xu system provides a motivation to incorporate intent in DAA systems, as explained in the following sections.

3.1.3 MDP for intent-based detect and avoid

Take two agents, labelled by $i \in \mathcal{I} = \{1, 2\}$, which move on the same horizontal plane according to

$$\dot{\mathbf{p}}_i \equiv \begin{bmatrix} \dot{x}_i \\ \dot{y}_i \\ \dot{\sigma}_i \end{bmatrix} = \begin{bmatrix} v_i \cos\sigma_i \\ v_i \sin\sigma_i \\ u_i + w_i \end{bmatrix}, \tag{10}$$

where $\mathbf{p}_i \in \mathbb{R}^3$ is the pose vector, which combines the position (x_i, y_i) and the heading angle σ_i, $v_i \in \mathbb{R}_{\geq 0}$ is the linear velocity , $u_i \in \mathbb{R}$ is the turning rate, and $w_i \in \mathbb{R}$ a stochastic disturbance. Agent $i = 1$ is the ownship and agent $i = 2$ is the intruder. Each agent is tasked with reaching a set-point point on their respective nominal trajectory with an intended pose $\mathbf{q}^T = [\bar{x}_i, \bar{y}_i, \bar{\sigma}_i]^T$, which is called intent. This information is shared among all agents, hence granting intent-awareness.

The discrete-time dynamics for a two-agent encounter can be represented by:

$$\mathbf{s}^{+} \equiv \begin{bmatrix} \rho_{1,2} \\ \theta_{1,2} \\ \sigma_{2,1} \\ \bar{\rho}_1 \\ \bar{\theta}_1 \\ \bar{\rho}_2 \\ \bar{\theta}_2 \end{bmatrix}^{+} \leftarrow \begin{bmatrix} \left\| \left[x_2^{+} - x_1^{+}, y_2^{+} - y_1^{+} \right] \right\|_2 \\ \operatorname{atan2}\left(y_2^{+} - y_1^{+}, x_2^{+} - x_1^{+} \right) - \sigma_1^{+} \\ \sigma_2^{+} - \sigma_1^{+} \\ \left\| \left[\bar{x}_1 - x_1^{+}, \bar{y}_1 - y_1^{+} \right] \right\|_2 \\ \operatorname{atan2}\left(\bar{y}_1 - y_1^{+}, \bar{x}_1 - x_1^{+} \right) - \sigma_1^{+} \\ \left\| \left[\bar{x}_2 - x_2^{+}, \bar{y}_2 - y_2^{+} \right] \right\|_2 \\ \operatorname{atan2}\left(\bar{y}_2 - y_2^{+}, \bar{x}_2 - x_2^{+} \right) - \sigma_2^{+} \end{bmatrix} \tag{11}$$

Here $\rho_{1,2}$ denotes the distance between the positions of the ownship (1) and intruder (2), $\theta_{1,2}$ denotes the relative bearing angle of the intruder w.r.t. the ownship (relative to the heading of the ownship), $\sigma_{2,1}$ denotes the relative heading of the intruder w.r.t. the ownship, $\bar{\rho}_1$ and $\bar{\rho}_2$ are the distances between the position and intended endpoint of the ownship and intruder, and $\bar{\theta}_1$ and $\bar{\theta}_2$ are the relative bearing angles of the endpoint w.r.t. to the current position of the ownship and intruder (relative to the heading). A visual representation of the two-agent aircraft encounter geometry with intent information for horizontal collision avoidance system is shown in Figure 5.

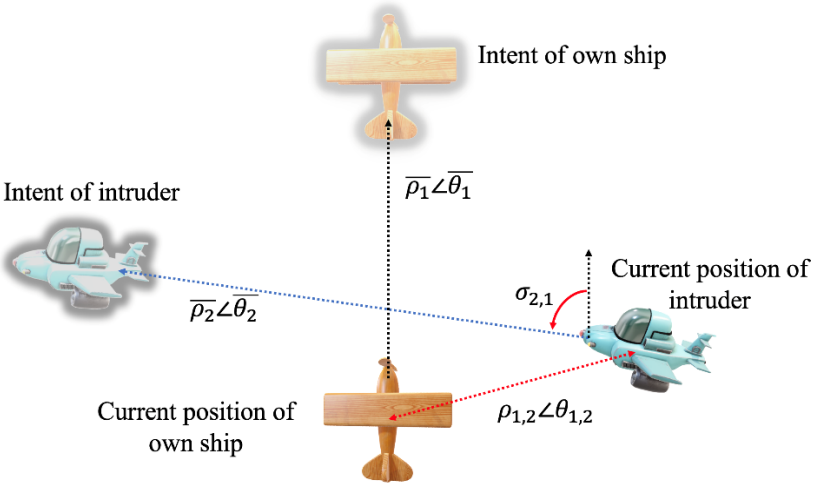

Figure 5. Aircraft encounter geometry with intent information for horizontal collision avoidance system

The MDP in Eq.(9) is used to find an optimal policy $\boldsymbol{\pi}^{\star}(\mathbf{s})$, which maps states to actions in a way that minimizes the cost. The cost function is based on two primary factors:

- Minimizing the risk of conflicts by keeping a safe distance from other aircraft.
- Ensuring that the aircraft reaches its destination efficiently, by following the shortest path.

A termination state is defined for when the aircraft reaches its destination. Upon reaching this state, no further cost is incurred, ensuring that the aircraft has completed its journey safely. The system assumes

that the transition probabilities, which represent the likelihood of moving from one state to another, remain constant over time (time-invariant). This simplifies the computation of the optimal policy, as the system does not need to account for changes in transition dynamics over time. In summary, by incorporating intent information into the MDP formulation, the DAA system can make better decisions, accounting not only for the current state of the aircraft but also for the future behavior of nearby aircraft. This approach leads to safer and more efficient flight paths.

3.2 Robust Model Predictive Control

3.2.1 Model Predictive Control

Model Predictive Control (MPC) is a powerful optimization-based control method that produces control policies by solving an optimal control problem at each discrete time instant based on the current system state $\mathbf{s}$, over a finite, receding horizon. As one of the main benefits, MPC can directly account for state-input constraints in optimization with computational efficiency, especially for deterministic systems. For deterministic dynamics, the problem is often formulated as:

$$\begin{aligned} &\min_{\hat{\mathbf{s}},\hat{\mathbf{a}}} \quad V_{\mathrm{f}}\left(\hat{\mathbf{s}}_N\right)+\sum_{k=0}^{N-1} L\left(\hat{\mathbf{s}}_k,\hat{\mathbf{a}}_k\right) \\ &\text{such that } \forall k\in\{0,\ldots,N-1\}: \\ &\quad \hat{\mathbf{s}}_{k+1}=\mathbf{f}_d\left(\hat{\mathbf{s}}_k,\hat{\mathbf{a}}_k\right),\quad \hat{\mathbf{s}}_0=\mathbf{s},\quad \mathrm{h}\left(\hat{\mathbf{s}}_k,\hat{\mathbf{a}}_k\right)\le 0,\quad \hat{\mathbf{a}}_k\in\mathcal{A} \end{aligned} \tag{12}$$

where L is the stage cost, V_{f} is the terminal cost, $\mathbf{f}_d\left(\hat{\mathbf{s}}_k,\hat{\mathbf{a}}_k\right)$ is the deterministic dynamics model, $\mathrm{h}\left(\hat{\mathbf{s}}_k,\hat{\mathbf{a}}_k\right)$ represents constraints, N is the horizon length and $\mathcal{A}$ is the set of control inputs. In order to distinguish between the actual system trajectory and the predicted state-input profile, we use the notation *hat* $\hat{\cdot}$ for the latter. The open-loop optimization of Eq.(12) is solved recursively at each state $\mathbf{s}$ and produces a complete profile of control inputs $\hat{\mathbf{a}}^*=\{\hat{\mathbf{a}}_0^*,\ldots,\hat{\mathbf{a}}_{N-1}^*\}$ and corresponding state predictions $\hat{\mathbf{s}}^*=\{\hat{\mathbf{s}}_0^*,\ldots,\hat{\mathbf{s}}_N^*\}$. The notation *star* $\cdot^*$ is used to refer to the optimal value of the decision variables. To incorporate feedback, only the first element $\hat{\mathbf{a}}_0^*$ of the input sequence $\hat{\mathbf{a}}^*$ is applied to the system, a successor state $\mathbf{s}^+$ is attained in the next time instance, and the optimization Eq.(12) is solved again for the new state $\mathbf{s}^+$. Therefore, MPC-based policy refers to the following policy:

$$\boldsymbol{\pi}_{MPC}(\mathbf{s})=\hat{\mathbf{a}}_0^* \tag{13}$$

A deterministic model, such as $\mathbf{f}_d\left(\hat{\mathbf{s}}_k,\hat{\mathbf{a}}_k\right)$ used in Eq.(12), does not capture stochastic uncertainties. Therefore, it is limited to situations where there are insignificant uncertainties and small disturbances in the system. Consequently, it is necessary to employ a notion of robust MPC for nonlinear systems that is computationally efficient. Nevertheless, based on the online receding horizon nature of the MPC, there are circumstances where traditional MPC can still perform acceptably, even in stochastic systems. This is often the case when the system uncertainties are relatively small, predictable, or can be well approximated by the deterministic model used by the MPC. In the following section, we will introduce an efficient and robust MPC approach.

3.2.2 Robust scenario-tree MPC

Scenario-tree MPC is a computationally tractable robust MPC for nonlinear systems. Consider m samples from a distribution W, denoted by $\mathbf{w}^i$, where superscript i denotes a scenario index. Suppose that a stochastic system is approximated by the following set of dynamics:

$$\mathbf{s}^+ = \mathbf{f}(\mathbf{s}, \mathbf{a}, \mathbf{w}^i), \quad i \in \{1, \ldots, m\}. \tag{14}$$

As such, there will be different trajectories based on the possible $\mathbf{w}^i$ at each time instance. The number of trajectories (scenarios) grows exponentially with the length of the MPC horizon. Accordingly, it is common to fix the uncertain parameters after a certain period of time called the robust horizon $N_r < N$. Figure 6 illustrates the evolution of the system represented as a scenario-tree for $m = 3$, $N_r = 2$, and $N = 3$. Each scenario corresponds to a specific trajectory, starting from the root (current state) to a leaf. Each node in the tree represents a possible state of the system at a future time step, and each branch represents a possible transition due to the uncertain parameter $\mathbf{w}^i$ with $i \in \{1, 2, 3\}$. The uncertain parameter is fixed to $\mathbf{w}^1$ after $N_r = 2$ steps.

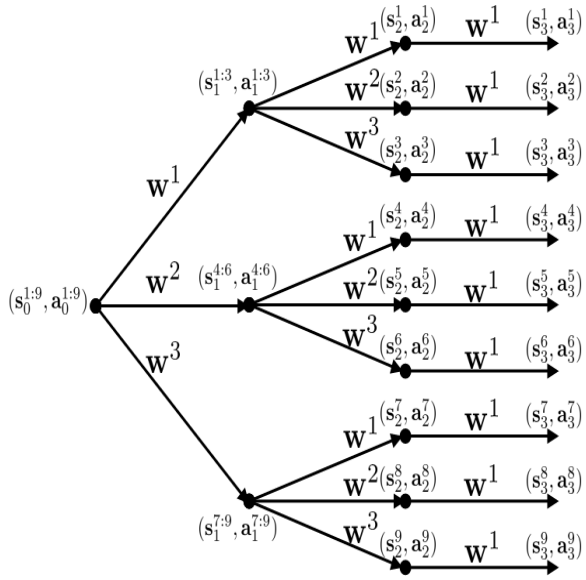

Figure 6. The evolution of the system represented as a scenario-tree.

The number of all scenarios is $M = m^{N_r}$; in Figure 6, there are 9 different scenarios. The superscripts in the states denote a specific scenario, while the subscripts are used for the time index. The notation $\mathbf{s}_k^{i:j}$ denotes $\mathbf{s}_k^i = \mathbf{s}_k^{i+1} = \ldots = \mathbf{s}_k^j$. For instance, in the beginning, all scenarios start with the current specific state that is denoted by $\mathbf{s}_0^{1:9}$. Furthermore, since realizations of uncertainty cannot be predicted, control actions depend solely on historical realizations. Hence, $\mathbf{a}_i^j = \mathbf{a}_i^l, \forall i = 0, \ldots, k$, if the uncertainty realization for scenarios j and l are identical up to and including time stage k. This restriction is commonly referred to

as a *non-anticipativity* constraint [50]. In Figure 6 this constraint can be expressed as $\mathbf{a}_0^{1:9}$, $\mathbf{a}_1^{1:3}$, $\mathbf{a}_1^{4:6}$, and $\mathbf{a}_1^{7:9}$.

Considering all scenarios in the constraints and optimizing the cost function based on their average, the scenario-tree MPC optimization can be expressed as follows:

$$\min_{\hat{\mathbf{s}},\hat{\mathbf{a}}} \quad \sum_{j=1}^{M}\left(V_{\mathrm{f}}\left(\hat{\mathbf{s}}_N^j\right)+\sum_{k=0}^{N-1} L\left(\hat{\mathbf{s}}_k^j,\hat{\mathbf{a}}_k^j\right)\right),$$

such that

$$\forall k\in\{0,\ldots,N-1\},\forall j\in\{1,\ldots,M\}:\quad \hat{\mathbf{s}}_{k+1}^j=\mathbf{f}\left(\hat{\mathbf{s}}_k^j,\hat{\mathbf{a}}_k^j,\mathbf{w}^i\right),\quad \hat{\mathbf{s}}_0^j=\mathbf{s},\quad \mathbf{h}\left(\hat{\mathbf{s}}_k^j,\hat{\mathbf{a}}_k^j\right)\le 0,\quad \hat{\mathbf{a}}_k^j\in\mathcal{A},$$

$$\forall l\in\{1,\ldots,M\},\forall i\in\{0,\ldots,k\}:\quad \hat{\mathbf{a}}_k^j=\hat{\mathbf{a}}_k^l \text{ if } \hat{\mathbf{s}}_i^j=\hat{\mathbf{s}}_i^l \tag{15}$$

where

$$i=\begin{cases}\mathrm{mod}\left(\left\lceil \dfrac{j}{m^{N_r-k-1}}\right\rceil,m\right) & k<N_r\\ 1 & k\ge N_r\end{cases} \tag{16}$$

is the index of parameter $\mathbf{w}^i$ for each time-scenario pair, and where the function $1\le \mathrm{mod}(n,m)\le n$ is the remainder of n/m if n/m is not an integer, otherwise $\mathrm{mod}(n,m)=m$. Moreover, $\lceil n\rceil := \min\{m\in\mathbb{Z}|m\ge n\}$.

Analogous to the standard MPC, the policy of robust MPC based on a scenario-tree is determined as follows:

$$\boldsymbol{\pi}_{\mathrm{RMPC}}\left(\mathbf{s}\right)=\hat{\mathbf{a}}_0^{1:M,\star}, \tag{17}$$

where $\hat{\mathbf{a}}_0^{1:M,\star}$ is the optimal solution of $\hat{\mathbf{a}}_0^1=\ldots=\hat{\mathbf{a}}_0^M$.

3.2.3 Intent-aware scenario-tree MPC-based DAA in the horizontal plane

In this section an intent-aware scenario-tree MPC-based DAA system for aircraft movements in the horizontal plane is designed. We consider two aircraft, ownship and intruder, to have the following equation of motion in the discrete-time setting:

$$\mathbf{s}_{k+1}^i:=\begin{bmatrix}x_{k+1}^i\\ y_{k+1}^i\\ \varphi_{k+1}^i\end{bmatrix}=\begin{bmatrix}x_k^i\\ y_k^i\\ \varphi_k^i\end{bmatrix}+t_e\begin{bmatrix}v_k^i\cos\varphi_k^i\\ v_k^i\sin\varphi_k^i\\ u_k^i\end{bmatrix},\quad i\in\{1,2\}, \tag{18}$$

where superscript $i=1$ stands for the ownship, $i=2$ corresponds to the intruder, $\mathbf{s}_k^i$ is the state of the aircraft i at time k, x_k^i and y_k^i are the position states, and φ_k^i is the heading angle. The constant t_e is

the sampling time and is assumed to be 1 sec, and v_k^i and u_k^i are the linear and angular velocities (speed and rate of turn), respectively. The linear and angular velocities are considered to be the control inputs.

It is assumed that the DAA system is aware of the intent of an intruder. Specifically, the intent is represented as a series of waypoints that the intruder will pass through. Using these waypoints, the overall path is determined based on the Dubins path, which is the optimal path that connects the intruder's current position to the next waypoint [51]. The Dubins path is tangent to the initial conditions (position and direction) and the next waypoint. It determines the path by specifying maneuvers such as turning left (L) by a certain angle, moving straight (S) for a specific duration, and finally turning right (R) by a certain angle. Figure 7 illustrates an example of such a path, denoted as LSR. The gray dashed-line circles represent the maximum possible curvature based on the limitations of linear and angular speed.

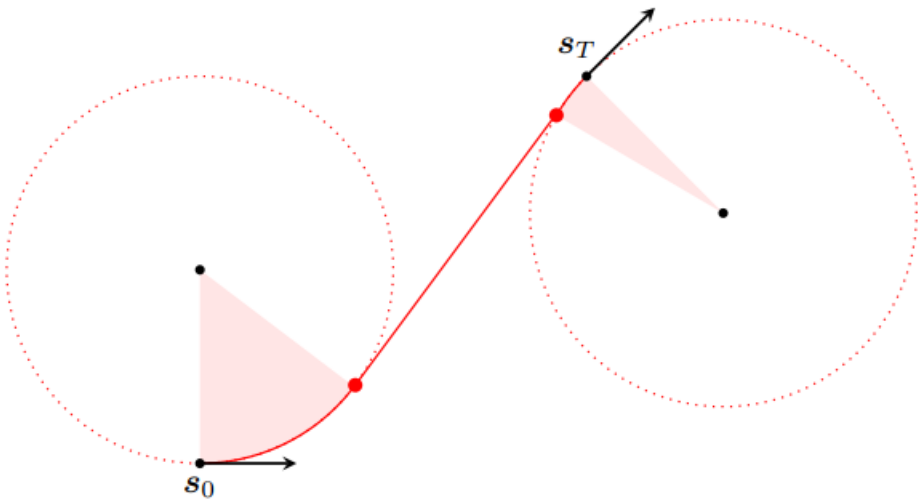

Figure 7. An LSR Dubin's path from an initial to a final state.

More specifically, the optimal Dubin's path provides a mapping from the initial-target state pair to the path connecting these two states, i.e., $(\mathbf{s}_0, \mathbf{s}_T) \rightarrow (\mathbf{s}_0, \mathbf{s}_1, \ldots, \mathbf{s}_T)$ and consequently the angular velocities can be obtained by a certain time-varying mapping, as follows:

$$u_k = D(\mathbf{s}_0, \mathbf{s}_T, k), \quad 0 \le k < T,$$

where D is a mapping from the initial state, the target state, and the time index k to the corresponding optimal angular velocity u_k, generating the optimal Dubin's path. This mapping can be obtained based on different types of optimal paths including RSR, RSL, LSR, LSL, RLR, or LRL. It has been assumed that the linear velocity is constant.

In the context of MPC, incorporating the intruder's intent information into the control synthesis for the ownship is relatively straightforward. First, the intruder's current state and waypoint are used to determine its Dubin path, which allows for the prediction of the intruder's upcoming control inputs. An MPC scheme is then formulated for the ownship with objectives such as minimizing control effort or reaching a

destination, while adhering to constraints such as maintaining a minimum safe distance from the intruder. Consequently, the intruder's dynamics and predicted control inputs are integrated into the ownship MPC scheme. By utilizing the intruder's intent information, these values can be accurately included in the ownship MPC, reducing uncertainty. In contrast, without knowledge of the intruder's intent, predictions are based on assumptions like a direct path for the intruder, which introduces more uncertainty.

For the ownship ($i=1$), we consider v_k^i and u_k^i as the control input. These velocities are assumed to be obtained based on Dubin's optimal path for the intruder ($i=2$). Therefore, based on the initial state and the intent state for the intruder, we can obtain the corresponding Dubin's path and the corresponding control inputs. Then, the aim is to steer the ownship to its intent state while maintaining sufficient separation with the intruder using an MPC method.

In order to incorporate uncertainties that arise from the intruder, when the ownship is at the state $\mathbf{s}_t$, $0 \le t \le T$, we utilize an intent-aware scenario-tree MPC approach as follows. The minimization concerns finding angular and linear velocities, such that the target state $\mathbf{s}_T$ is achieved while variations in angular velocity are minimized:

$$\min_{u^1_{0:N-1}, v^1_{0:N-1}, \mathbf{s}^1_{0:N}} (\mathbf{s}^1_N - \mathbf{s}_T)^\top Q_\mathrm{f} (\mathbf{s}^1_N - \mathbf{s}_T) + \sum_{k=0}^{N-1} (\mathbf{s}^1_k - \mathbf{s}_T)^\top Q (\mathbf{s}^1_k - \mathbf{s}_T) + R(u^1_k - u^1_{k-1})^2 \tag{19}$$

where Q_f and Q are positive definite matrices, R is a positive constant. The minimization is achieved in a receding horizon manner given the following constraints and assumptions for M different scenarios, i.e. $\forall j \in \{1, \ldots, M\}$, and along timesteps, i.e. $\forall k \in \{0, \ldots, N-1\}$:

1. The dynamics of the ownship ($i=1$) and the dynamics of the intruder for M different scenarios $i \in \{2, j\}, j \in \{1, \ldots, M\}$ are according to the horizontal movement of Eq.(18);
2. The horizontal distance between the aircraft is never below a minimum separation distance ρ: $\rho \le \sqrt{(\mathbf{s}^1_k - \mathbf{s}^{2,j}_k)^\top \mathrm{diag}(1,1,0)(\mathbf{s}^1_k - \mathbf{s}^{2,j}_k)}$;
3. The linear and angular velocities of the ownship are between minimum and maximum values: $\underline{u}^1 \le u^1_k \le \bar{u}^1, \quad \underline{v}^1 \le v^1_k \le \bar{v}^1$;
4. The scenario-tree MPC uses three branches ($m=3$) to represent uncertainty in the angular velocity of the intruder. The angular velocity is assumed to be minimum $\underline{u}^2$, maximum $\bar{u}^2$, or such that he optimum Dubin's path is followed between the intruder waypoints, until the robust horizon N_r, while it is assumed that it follows its optimum Dubin's path after the robust horizon:

$$
\begin{aligned}
&\text{if } k < N_r: \\
&\qquad u_k^{2,j} = \begin{cases} \bar{u}^2 & \text{if mod}\left(\left\lceil \frac{j}{3^{N_r-k-1}} \right\rceil, 3\right) = 0 \\ \underline{u}^2 & \text{if mod}\left(\left\lceil \frac{j}{3^{N_r-k-1}} \right\rceil, 3\right) = 1 \\ D(\mathbf{s}_0^2, \mathbf{s}_T^2, t+k) & \text{if mod}\left(\left\lceil \frac{j}{3^{N_r-k-1}} \right\rceil, 3\right) = 2 \end{cases} \\
&\text{if } k \geq N_r: \\
&\qquad u_k^{2,j} = D(\mathbf{s}_0^2, \mathbf{s}_T^2, t+k)
\end{aligned}
\tag{20}
$$

5. The linear velocity of the intruder is assumed to be constant: $v_k^{2,j} = \bar{v}^2$;
6. The initial state of the ownship in the MPC $\mathbf{s}_0^1$ is set to the actual current state of the ownship $\mathbf{s}_t$: $\mathbf{s}_0^1 = \mathbf{s}_t$.

In other words, these M scenarios account for uncertainties in the intruder's execution of angular velocity (i.e., control input) by considering both the nominal angular velocities that follow the optimal Dubin's path and the worst-case angular velocities over a few steps ahead (robust horizon). Incorporating these scenarios in the optimization reduces the feasibility domain, leading to more conservative control inputs for the ownship compared to the situation when only the intruder's nominal trajectory is considered. As such, this approach enhances robustness against potential uncertainties arising from e.g., human error, mismatches in the intent information, external disturbances, etc.

In cases where intent information is not available or accounted for, a straight-line prediction is typically assumed for the intruder. As such, in Eq.(20) of the MPC the value of the function D is replaced with zero. This discrepancy between the prediction and the actual path of the intruder can result in a non-optimal path. It is important to note that considering intent information is beneficial not only for optimization and safety purposes but also for the MPC recursive feasibility. Recursive feasibility in MPC schemes can be challenging when the system is uncertain, such as when the intruder's waypoint does not align with the predicted path. Knowledge of the intruder's intent assists in reducing the uncertainty and, consequently, achieving recursive MPC feasibility.

3.3 A* path planning-based conflict resolution

3.3.1 Conflict risk cost function

In [34] a path planning approach was used for conflict resolution between sets of aircraft using horizontal manoeuvring. In this section the method is described and extended. A set of N aircraft flying in a horizontal plane is considered. Each aircraft i has horizontal position (x_i, y_i) and heading θ_i (0 deg is positive x-axis, 90 deg is positive y-axis), and it flies with a constant speed v_i along a set of waypoints $\mathbf{w}_i$. A cost function $c_{i,j}^{CR}$ is defined for the risk of a conflict of an ownship i with an intruder j as

$$c_{i,j}^{CR} = \begin{cases} k_1 \exp\left(-\left(\left(\frac{\left(x_i - x_j\right)\cos\theta_j + \left(y_i - y_j\right)\sin\theta_j}{\sqrt{2}\sigma_j^a} \right)^2 + \left(\frac{-\left(x_i - x_j\right)\sin\theta_j + \left(y_i - y_j\right)\cos\theta_j}{\sqrt{2}\sigma_j^c} \right)^2 \right) \right) & \text{if } d_{i,j} > D \\ \infty & \text{if } d_{i,j} \leq D \end{cases} \tag{21}$$

where σ_j^a and σ_j^c represent the standard deviations of along-track and cross-track position uncertainties of the intruder, k_1 is a tuning parameter, $d_{i,j}$ is the horizontal distance between the aircraft, and D is a (strict) separation criterion. The cost is always positive, it increases for smaller distances, depending on the along-track and cross-track position uncertainties of the intruder, and it represents a hard constraint for distances smaller than the separation criterion D. The conflict risk cost function may be adapted, for example to represent other threats, such as closed airspace or bad weather zones, or to use different tuning of the conflict risk.

The method can account for multiple intruders. In such case the total conflict risk cost of the ownship is the sum of the costs with respect to all intruders:

$$c_i^{CR} = \sum_{j \neq i} c_{i,j}^{CR} \tag{22}$$

3.3.2 A* path planning per aircraft

The principal idea of the conflict resolution approach is to formulate it as a path planning problem, which minimizes the total cost of a flight path towards a goal by considering both the distance and the total conflict risk cost along the path. This is done using the A* path solving algorithm [52]. Since A* works using a graph of *nodes* and *edges*, flight paths need to be represented as such. Using a discrete time step t_d a flight path is converted into a set of nodes that are connected by edges. To allow the aircraft to turn, neighbouring nodes are added to each node, which represent possible turns the aircraft can take. The number of neighbours leaving a node is n_θ. Useful values are $n_\theta = 3$, representing left, straight, right, or $n_\theta = 5$, representing strong left, weak left, straight, weak right, strong right. The associated angles are determined by the maximum turn rate in $°/s$ multiplied by t_d, with additional angles spaced evenly between for larger values n_θ. Figure 8 illustrates a node network for horizontal manoeuvring.

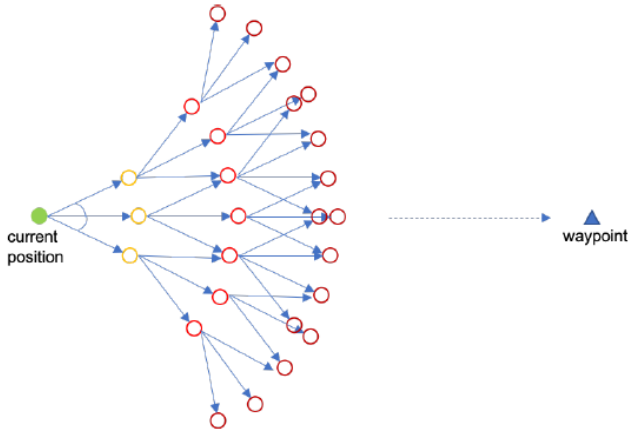

Figure 8. Nodes and edges for path finding for horizontal manoeuvring, source [34]

The A* search algorithm is an informed best-first search algorithm, which uses an evaluation function

$$f(n) = g(n) + h(n) \tag{23}$$

where $g(n)$ is the path cost from the initial state to node n, and $h(n)$ is a heuristic function for the *estimated* cost of the shortest path from n to a goal state [52]. The A* algorithm is complete, meaning that it is guaranteed to find a solution when there is one. A* is cost-optimal (i.e. it finds a solution with the lowest costs) if the heuristic function is admissible. An admissible heuristic means that it never overestimates the cost to reach a goal. With an inadmissible heuristic, A* may or may not be cost-optimal. A consistent heuristic means, that the single number $h(n)$ is less than the sum of the cost $c(n,a,n')$ of the action from n to n' plus the heuristic estimate $h(n')$. A* with a consistent heuristic is optimally efficient, in the sense that any algorithm that extends search paths from the initial state, and uses the same heuristic information, must expand all nodes that are surely expanded by A* [52].

The evaluation function in the conflict resolution approach of [34] has been chosen such that it does not well include costs for path length, and such that the heuristic function may be inadmissible and inconsistent. Next we propose a more suitable evaluation function, which takes into account these characteristics.

As a start, we define aircraft-specific costs of a flight trajectory element c_i^{FT}. These are the costs of the aircraft to traverse during a time t_d to a next node in the network. Higher costs of a flight trajectory element may reflect priority of a flight, e.g. due to a medical emergency condition or a lack of fuel / remaining battery.

The search for an optimal path for aircraft i given the total conflict risk cost c_i^{CR} and the flight trajectory element cost c_i^{FT} can be achieved by the following path cost and heuristic functions:

$$\begin{aligned} g_i(n) &= g_i(n-1) + c_i^{FT} + c_i^{CR} \\ h_i(n) &= k_2 c_i^{FT} \frac{\left\| \mathbf{x}_i(n) - \mathbf{x}_i(G) \right\|}{t_d v_i} \end{aligned} \tag{24}$$

Herein the path cost is increased by the flight trajectory element cost plus the total conflict risk. The positive factor k_1 in Eq.(21) allows to provide more emphasis on the conflict risk, in particular larger values lead to more risk adverse paths. The heuristic function determines the minimum flight trajectory costs by multiplying the number of steps needed to pass the distance between the aircraft position at the current node with respect to its goal G by the flight trajectory element cost c_i^{FT}, and it weights this value with a positive factor k_2. If $k_2 = 1$ then the heuristic is admissible, thus assuring the search to be cost-optimal. However setting $k_2 > 1$ may reduce the solution space and thus accelerate the search, while still providing acceptable results.

3.3.3 Path planning for a set of aircraft

The A* path planning approach outlined in above section finds an optimal flight trajectory for a single flight under the condition that there are a number of intruders that need to be avoided and that do not adapt their trajectories. So the intruders are considered as obstacles with fixed trajectories. A next step is to achieve a multi-agent optimization that adapts the trajectories of multiple aircraft. Such multi-agent optimization typically has high computational complexity [53]. In [34, 35] the multi-agent optimization is achieved by iterations of a series of ordered single-agent A* optimizations, where one the aircraft is optimized at the time given fixed trajectories of other aircraft. In [34] for horizontal movements the order is determined such, that the aircraft pairs with the earliest time to loss of separation are resolved first, if these times are equal, the aircraft with lower cost to change flight path are placed later in the sequence order, and if priority cannot be determined a random order is set. The sensitivity for the ordering strategy was not analysed in [34, 35].

Pseudocode for the multi-agent A* path planning optimization is as follows:

$$
\begin{aligned}
&\text{Determine costs } g_i(\mathbf{x}_i^0(n))\ \forall i,\ n = 1 \ldots N_i \text{ for initial trajectories } \mathbf{x}_i^0 \\
&\text{IF RiskExceedance}(g_i(\mathbf{x}_i^0(n)), \forall i,\ n = 1 \ldots N_i) \text{ THEN} \\
&\quad order \leftarrow \text{DetermineOrder}(g_i(\mathbf{x}_i^0(n)), \forall i,\ n = 1 \ldots N_i) \\
&\quad \text{WHILE NotConverged,} \\
&\qquad \text{FOR } i \text{ in } order \text{ DO } \mathbf{x}_i^k \leftarrow A^*(\mathbf{x}_i^{k-1} \mid \mathbf{x}_{\forall j \neq i}) \\
&\quad \text{ENDWHILE} \\
&\quad \text{IF RiskExceedance}(g_i(\mathbf{x}_i^k(n)), \forall i,\ n = 1 \ldots N_i) \text{ THEN} \\
&\qquad \text{NoSolution} \\
&\quad \text{ELSE} \\
&\qquad \text{Solution} \leftarrow \mathbf{x}_i^k(n)\ \forall i,\ n = 1 \ldots N_i \\
&\quad \text{ENDIF} \\
&\text{ENDIF}
\end{aligned}
\tag{25}
$$

It firsts determines path costs for the all initial trajectories (e.g. straight routes from start to end), including the conflict risk costs and flight trajectory element costs. Next it determines if there is an exceedance of

the risk for the initial trajectories. If so the multi-agent iterative A* path planning is initiated. First an order in the sequence of aircraft is determined. This order can be based upon the path costs, but is can also be some random order. For this order the trajectory of each aircraft is adapted using A*, while the trajectories of other aircraft are fixed. The iterations continue until the differences in the costs are sufficiently small according to some convergence criterion. Finally it is checked if the final risk is acceptable or not. The latter case implies that no acceptable solution has been achieved.

4 Models for evaluation by sets of encounters

Encounter scenarios describe the characteristics of encounters between a number of aircraft. Sets of encounter scenarios can be used to evaluate the performance of DAA systems, and they can be means for the optimization of DAA systems. This chapter describes the models of encounter scenarios in support of the aviation use case in SymAware. Section 4.1 describes input of DAA systems. Section 4.2 describes the closed-loop delay for responding to DAA advisories. Section 4.3 defines the geometry of close encounters of pairs of drones.

4.1 Detect and avoid system

It is assumed that a detect and avoid (DAA) system in aircraft *i* that is involved in an encounter with aircraft *j* has access to state-based information and potentially to intent-based information.

State-based input

The DAA system of the ownship aircraft *i* always has access to state-based information, which is based on measured positions and ground speeds of the ownship and the intruder aircraft *j* available in the FMS:

$$\begin{aligned}
\mathbf{s}_{t,FMS,i}^{AC,i} &= \mathbf{s}_{t,AC,i} + \boldsymbol{\varepsilon}_t \\
\mathbf{s}_{t,FMS,i}^{AC,j} &= \mathbf{s}_{t,AC,j} + \boldsymbol{\varepsilon}_t \\
\mathbf{v}_{t,FMS,i}^{g,AC,i} &= \mathbf{v}_{t,AC,i}^{g} + \boldsymbol{\varepsilon}_t \\
\mathbf{v}_{t,FMS,i}^{g,AC,j} &= \mathbf{v}_{t,AC,j}^{g} + \boldsymbol{\varepsilon}_t
\end{aligned} \tag{26}$$

where $\boldsymbol{\varepsilon}_t$ indicates dynamic errors, which depend on the measurement process. The surveillance data of aircraft *j* as acquired by aircraft *i* may be obtained by cooperative surveillance, where data is actively shared via a communication link, or it may be acquired by an independent surveillance process (like a radar). The applied means of surveillance has impact on the updating frequency, delays and error characteristics.

Intent-based input

The DAA system of aircraft *i* may have access to intent-based information on planned waypoints of aircraft *i* and *j*: $\mathbf{q}_{t,FMS,i}^{FP,i}$ and $\mathbf{q}_{t,FMS,i}^{FP,j}$, where the intent information of the intruder aircraft *j* is shared via a data communication link. The waypoints that were planned at the start of a flight may have been extended by additional intermediate waypoints as advised by the DAA systems of aircraft *i* and *j*.

4.2 Response delay and remote pilot

Flights may be operated autonomously or they may be operated by a remote pilot. In autonomous operations the flights are only controlled by the FMS, based on the flight plan and DAA advisories. In remotely piloted operations the evolution of the flights is impacted by the interpretation of the DAA advisories by a remote pilot model and the decision-making.

It is assumed that both in autonomous operations and in remotely piloted operations the DAA advisories are followed. The only difference that is accounted for is a closed-loop delay between the time of DAA

advisories and their implementation in flight control. This delay is small and can be considered constant for autonomous operations, while it is larger and may be chosen from a probability distribution for remotely piloted operations. In remotely piloted operations the closed loop delay includes components the uplink and downlink of data, for the processing of data in the remote pilot station, for the processing time by the remote pilot, and possibly for coordination with air traffic control.

A closed-loop delay $\tau_{t,i}^{d}$ for the control loop of aircraft *i* is implemented by a time shift of the DAA advisories between the time that the advice is provided by the DAA system and the time that the advice is used by the FMS for mission and flight control of aircraft *i*.

4.3 Closest point of approach in encounters with straight trajectories

In support of the analysis and design of detect and avoid systems close encounters of drones will be designed. Encounters where both drones are flying with constant speeds along straight trajectories are considered:

$$\begin{aligned} \mathbf{s}_{t,i} &= \mathbf{s}_{0,i} + \mathbf{v}_i t \\ \mathbf{s}_{t,j} &= \mathbf{s}_{0,j} + \mathbf{v}_j t \end{aligned} \tag{27}$$

It is assumed that the aircraft have a closest point of approach (CPA) at time $t = 0$ when aircraft *i* is at the origin $\mathbf{s}_{0,i} = (0,0,0)$ with particular vertical and horizontal miss distances $d_{i,j}^{VMD}$ and $d_{i,j}^{HMD}$.

The distance between the aircraft is

$$d_{i,j,t} = \left\| \mathbf{s}_{t,i} - \mathbf{s}_{t,j} \right\| = \left\| \mathbf{s}_{0,i} - \mathbf{s}_{0,j} + (\mathbf{v}_i - \mathbf{v}_j)t \right\| \tag{28}$$

The CPA being attained at $t = 0$ implies that $\partial d_{i,j,t} / \partial t = 0$ at $t = 0$:

$$\left. \frac{\partial d_{i,j,t}}{\partial t} \right|_{t=0} = 2(\mathbf{s}_{0,i} - \mathbf{s}_{0,j}) \cdot (\mathbf{v}_i - \mathbf{v}_j) = 0 \tag{29}$$

Using $\mathbf{s}_{0,i} = (0,0,0)$ leads to the following condition:

$$s_{0,j}^{x}(v_{0,i}^{x} - v_{0,j}^{x}) + s_{0,j}^{y}(v_{0,i}^{y} - v_{0,j}^{y}) + s_{0,j}^{z}(v_{0,i}^{z} - v_{0,j}^{z}) = 0 \tag{30}$$

Furthermore we have the conditions:

$$\sqrt{(s_{0,j}^{x})^2 + (s_{0,j}^{y})^2} = d_{i,j}^{HMD} \tag{31}$$

$$\left| s_{0,j}^{z} \right| = d_{i,j}^{VMD} \tag{32}$$

Eq.(32) implies

$$s_{0,j}^{z} = \pm d_{i,j}^{VMD} \tag{33}$$

Next, the determination of the positions in the horizontal plane distinguishes the following three cases.

Case 1

If $v_{0,i}^{x} = v_{0,j}^{x}$ and $v_{0,i}^{y} \neq v_{0,j}^{y}$ (in particular, the aircraft are flying in the same direction with different ground speeds, or they are flying in opposite direction), then

$$s_{0,j}^{y} = -s_{0,j}^{z} \frac{v_{0,i}^{z} - v_{0,j}^{z}}{v_{0,i}^{y} - v_{0,j}^{y}}$$
$$s_{0,j}^{x} = \pm\sqrt{(d_{i,j}^{HMD})^2 - (s_{0,j}^{y})^2} \tag{34}$$

Case 2

If $v_{0,i}^{x} = v_{0,j}^{x}$ and $v_{0,i}^{y} = v_{0,j}^{y}$ (in particular, the aircraft are flying in the same direction with same ground speed) then two kinds of solutions exist.

a) If $v_{0,i}^{z} = v_{0,j}^{z}$ then any $\mathbf{s}_{0,j}$ can be chosen such that they fulfil conditions (31) and (32). This is the case that the aircraft fly parallel with equal speeds and thus always are at the indicated HMD and VMD. It is not a realistic scenario.
b) If $v_{0,i}^{z} \neq v_{0,j}^{z}$ and $s_{0,j}^{z} = 0$ then values of $s_{0,j}^{x}$ and $s_{0,j}^{y}$ can be chosen such that they fulfil condition (31). This represents the case that the trajectories cross vertically (VMD=0) with a particular HMD.

Case 3

If $A \equiv (v_{0,i}^{x} - v_{0,j}^{x}) \neq 0$ then

$$s_{0,j}^{x} = -\frac{s_{0,j}^{y}(v_{0,i}^{y} - v_{0,j}^{y}) + s_{0,j}^{z}(v_{0,i}^{z} - v_{0,j}^{z})}{v_{0,i}^{x} - v_{0,j}^{x}} = -\frac{s_{0,j}^{y}B + C}{A} \tag{35}$$

$$\left(\frac{s_{0,j}^{y}B + C}{A}\right)^2 + (s_{0,j}^{y})^2 = (d_{i,j}^{HMD})^2 \tag{36}$$

with $B \equiv (v_{0,i}^{y} - v_{0,j}^{y})$ and $C \equiv s_{0,j}^{z}(v_{0,i}^{z} - v_{0,j}^{z})$. This can be rewritten as:

$$(A^2 + B^2)(s_{0,j}^{y})^2 + 2BCs_{0,j}^{y} + C^2 - A^2(d_{i,j}^{HMD})^2 = 0 \tag{37}$$

The solution of this quadratic equation is

$$s_{0,j}^{y} = \frac{-BC \pm A\sqrt{C^2 + (A^2 + B^2)(d_{i,j}^{HMD})^2}}{A^2 + B^2} \tag{38}$$

Typically $C^2 + (A^2 + B^2)(d_{i,j}^{HMD})^2 > 0$, meaning that there are two possible solutions for $s_{0,j}^{y}$. Next the value for $s_{0,j}^{x}$ follows from Eq.(36).

5 Methods for risk assessment in a simulation environment

The last chapter described sets of encounter scenarios, which allow to compare the performance of DAA systems for a number of encounter geometries and sources of variability. A different view can be obtained by a risk assessment of an operational concept, where DAA systems are applied in the context of various types of operations, variability in operations, and non-nominal operating modes. Scope, agent-based models and risk assessment methods for such evaluation are described in this chapter.

5.1 Scope

5.1.1 Concept of operation

The concept of operations considers an urban area, where vertical take-off and landing (VTOL) air taxi operations are performed between suburbs, and where en-route drones are operated that cross the area at a constant altitude. Operations may be controlled by pilots-in-command (PICs), who reside in remote pilot stations (RPSs), or they may be performed autonomously without a human operator. A command-and-control (C2) link system is used to send mission control data to the aircraft and to receive flight data. Navigation data is based on a global navigation satellite system (GNSS) and a pressure altimetry system. Surveillance data is based on transmission of automatic dependent surveillance broadcast (ADS-B) data to/from other aircraft. Each aircraft has a DAA system, which uses ownship navigation data and othership surveillance data (state-based data) and which may use future waypoints (intent-based data) to provide alerts and guidance to remain well clear of other aircraft.

5.1.2 Variability and nonnominal modes

Safety risk assessment of new concepts and technologies needs to assess (1) how effective the new concepts and technologies are if they work as intended, as well as (2) what risks are induced if elements in the new concepts and technologies are failing. The first category refers to variability/uncertainty that exists in every operation, such as normal errors in GNSS-based position estimates, delays in C2 link transmission, delays in response to DAA alerts by a PIC, and variations in speeds and accelerations. The second category refers to failure/off-nominal conditions that may occur during an operation, such as engines failure, C2 link system not working, or PIC not responding to DAA alerts. Such failure conditions typically have low occurrence frequencies and may exist for particular (random) durations. They can be distinguished in local failure conditions affecting a single flight, e.g. a failure of a C2 link system in a drone, and global failure conditions affecting multiple flights, e.g. a failure of C2 link ground infrastructure in a region. An overview of the normal variability and the failure modes that are in the scope is provided in Table 2. In the agent-based dynamic risk modelling approach followed in this study, the implications of normal

variability/uncertainty and (local/global) failure conditions are assessed in union. This allows to evaluate the normal performance of operations, as well as the implications of particular failure conditions in combination with otherwise normal variability in operations.

Table 2. Normal variability and failure modes in the scope of the models

Normal variability	Failure modes
Wind speed	Adverse weather not predicted
Customer demand: timing, locations	Wrong altitude in flight planning
Errors in altitude and vertical speed measurement	Engines failure
Errors in GNSS-based horizontal position and speed estimates	Reduced accuracy of pressure altimetry
Delay in ADS-B transmission (between drones)	GNSS-based estimation is not working (aircraft system or in whole region)
Delay in C2 link transmission (with RPS)	Reduced accuracy of GNSS-based estimation (single aircraft or whole region)
Delay in response to DAA alert by PIC	C2 Link not working (aircraft system, RPS, or whole region)
Rates of turn and climb/descent by PIC in DAA response	ADS-B system of aircraft is not working
Rates of climb, descent, turn, acceleration, deceleration, cruise speed during nominal flight	DAA system of aircraft is not working
	No/limited response to DAA alert by PIC

5.2 Agent-based modelling

5.2.1 Overview of the agent-based model

A high-level overview of the agent-based model is provided in Figure 9. It shows a number of unmanned aircraft that reside in a particular environment and airspace design. The UA movements are influenced by weather conditions and weather is being forecasted by meteorological services. There are customers who pose demand for missions. UA operators develop flight plans based on the mission demand, weather forecast and airspace layout. The flight plans are used by PICs as a basis to control the missions. The mission control commands by each PIC are uplinked by the C2 link to the flight management system (FMS) of the UA and this determines the flight performance (movements of the aircraft). The FMS also uses ownship state estimation data, consisting of GNSS-based position and speed estimates and pressure altimetry. This ownship data is shared with other UAs via ADS-B. The ownship and othership states data

are used by the DAA system on board of each UA to generate alerts and guidance advisories. The DAA output is downlinked by C2 link and shown on the traffic display in the RPS to the PIC. In response the PIC can adapt the mission to avoid close proximity to another drone. Alternatively, there may be an automatic response to DAA advisories, allowing autonomous adaptation of the flight profile. Details on the agents are provided next.

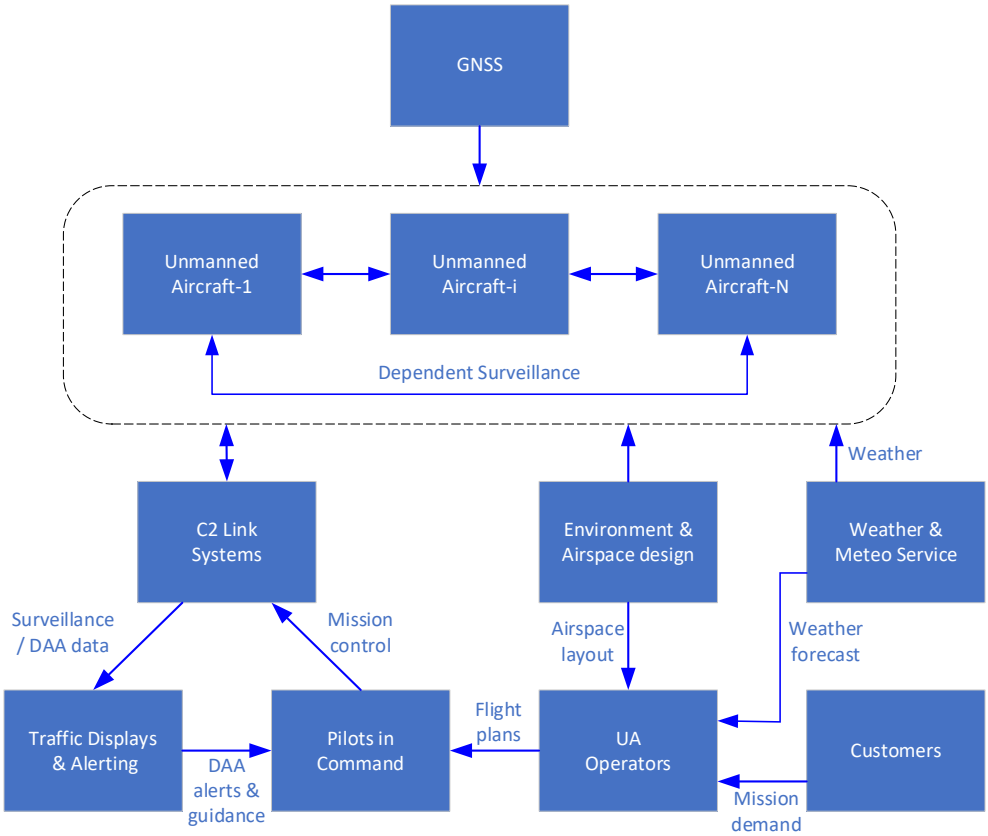

Figure 9. High-level overview of the agent-based model, from [54]

Weather and meteorological services

The modelling of weather and meteorological services represents a constant and uniform wind that can be set and it includes a random possibility of an adverse weather condition in the region that is not predicted by the meteorological services. The consequence of adverse weather is that the drone trajectories exhibit oscillatory motion around a nominal flight path, thus representing turbulence.

Customers

The entity Customers represents the demand for services from the UA operators for the mission types. This demand determines the traffic density in the simulations. The demand for the missions is represented by Poisson processes with mission-type dependent waiting times. It also determines mission-type dependent random departure and arrival positions in the urban areas or at the edges of flight zones.

UA operator

The UA operator performs the flight planning of the drone flights. The operator uses the airspace design and the customer demand as a basis for the flight planning. Flights are planned directly following the customer demand, there is no restriction in the number of available aircraft. The flight planning depends on the type of operation associated with the mission. For air taxi and en-route missions the shortest routes between start and end points are planned. The planning of flights sets a flight level that is between the altitude bounds of the airspace design for the type of operation considered. The operator can make an altitude planning error, which implies that a flight is planned at an altitude layer above or below the layer according to the airspace design.

UA flight performance

A number of aircraft types are defined, which are associated to the operation types (air taxi, and en-route UAS). The flight performance of the aircraft is specified, describing variables like position, heading, speed, and climb speed during manoeuvring. The input for the manoeuvring of the aircraft stems from the flight control system, which uses mission control settings for various flight phases. The flight performance model includes non-nominal modes for uncommanded motion of the drone in adverse weather and for engine failure.

Flight management system

The FMS of the aircraft contains the flight plan, ownship state estimates, received othership state estimates, output of the DAA system, and input from the PIC via the C2 link. This data is used for mission and flight control, as input of the DAA system, for ADS-B Out transmission to other aircraft, and to inform the PIC. The mission control system uses settings by the PIC to control the various flight phases of nominal operations and to change the trajectories in response to DAA alerts and guidance. Furthermore the mission control system includes autonomous control modes for contingency plans in the case of a lost C2 link or in the case of lost GNSS ownship estimation. The mission control system sends commands to the flight control system for the control of heading, air speed and altitude.

Ownship state estimation

The ownship state estimation includes GNSS-based estimation of horizontal position and speed and pressure altimetry for estimates of the altitude and vertical speed.

- The model for GNSS-based state estimation includes availability modes, representing working or not, and accuracy modes, representing the state estimation being in normal accuracy or reduced accuracy ranges. These modes are determined by modes at the GNSS receiver in each UA and modes representing the GNSS in the region. Errors in the position and speed estimates are chosen from Gaussian distributions with mode-dependent standard deviations.
- A model for the pressure altimetry system is used, which includes modes for the altitude and vertical speed estimation in normal accuracy ranges and for reduced accuracy ranges. Altitude and vertical speed measurement errors are chosen from Gaussian distributions with mode-dependent standard deviations.

Othership state data

Position and speed estimation data is exchanged with other UAs by ADS-B, thus providing the basis for dependent surveillance at each UAS. A model for the availability of the ADS-B system at each UAS is included, which represents the possibility of the system not being available. Transmission between a pair

of UASs requires the ADS-B at both UASs to be working. The duration of ADS-B transmission is chosen from a uniform distribution.

C2 link

The C2 link is the logical connection used for the exchange of information between the PIC in the RPS and the UA. A model for the availability of the C2 link system represents the possibility of the system not being available. It consists of three components, representing availability of C2 link systems in the UA and RPS, and availability of the C2 link infrastructure in the whole region. The latter component represents a global failure mode, affecting all UAs in the region. If C2 link transmission is possible, durations for the uplink and the downlink of information are chosen from uniform distributions.

DAA system

In the scope of the agent-based model, the DAA system is the prime means for detecting conflicts and providing guidance and alerts for remaining well clear. The DAA system may use state-based information only, such as in DAIDALUS, or it may in addition use intent-based data, such as in the DAA systems considered in SymAware.

Remote pilot station

The Remote Pilot Station receives aircraft data and DAA data via the C2 Link and shows this data to the PIC by traffic display and alerting.

Pilot in command

The PIC is the agent who sets the control of the various types of aircraft operations in nominal conditions and who is informed by the DAA system about conflicting aircraft and guidance to avoid close encounters. The situation awareness model of the PIC includes information of the ownship for nominal flight control actions, such as the flight plan, airspeed, altitude and heading, as well as the downlinked DAA data. Based on the situation awareness components, the PIC implements actions in response to the DAA alert and guidance information. PIC action models include: (1) PIC response mode, describing probabilities of response to horizontal and vertical guidance; (2) delay in the response to DAA alert and guidance information, chosen from a log-normal distribution; and (3) PIC mission control actions, which describe the manoeuvring to selected directions/altitudes and returning back to the planned trajectory if there is no DAA alert remaining.

5.2.2 Modelling of nominal and off-nominal conditions

For simulation-based safety risk assessment of operations it is needed to represent variability in performance both in nominal conditions and in off-nominal conditions. For this purpose the agent-based model includes a range of discrete states that describe nominal and off-nominal modes that influence particular characteristics of system performance. The system performance at a particular aircraft *i* may be influenced by local systems at the aircraft as well as by global systems outside of the aircraft (which also influence systems of other aircraft).

A schematic diagram of these elements of an agent-based model is shown in Figure 10. It shows two characteristics for local and global systems, which all have a nominal mode Working and an off-nominal mode Failing. Transitions between the modes are via delay gates, where the delays are chosen from exponential distributions. For instance, this model structure can describe the performance of GNSS-based state estimation at an aircraft. Here, characteristic A may represent availability for a local system (GNSS

receiver at aircraft *i*) and for a global system (GNSS performance in the region). Both the local and the global systems need to be in the Working mode for GNSS-based state estimation at aircraft *i*. Lack of GNSS-based state estimation implies that position estimates are not available for navigation and surveillance functions. Characteristic B in Figure 10 may represent the level of accuracy of GNSS-based state estimation as achieved by the local and global systems. Here the Working mode may represent a normal accuracy level, where state estimates are affected by normal errors, and the Failure mode may represent a reduced accuracy mode with enlarged errors in the state estimates. The achieved accuracy affects the navigation and surveillance performance of the affected aircraft.

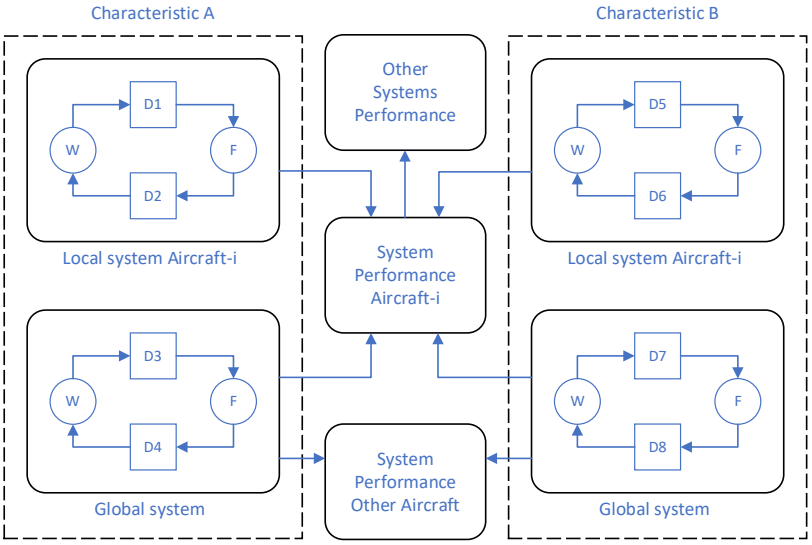

Figure 10. Schematic diagram of Working and Failing modes of local and global system characteristics A and B influencing system performance of aircraft i. D1 to D8 are transitions with exponentially distributed delays. Source: [54]

5.3 Rare event Monte Carlo simulation

Straightforward MC simulation of the dynamic and stochastic behaviour of the interacting agents only allows to assess probabilities of close proximities and collisions up to a certain level in a given time. In this study we use two techniques to accelerate the MC simulation: Interacting Particle System (IPS) method and risk decomposition for global failures.

5.3.1 IPS MC simulation

In [55, 56] acceleration of MC simulation has been achieved for air traffic scenarios using the IPS method of [57]. It makes use of a series of decreasing miss distances $d_{c+1} < d_c$ $(c = 1 \ldots M)$. Each miss distance d_c is composed of a horizontal miss distance (HMD) and a vertical miss distance (VMD). In IPS MC simulation, N runs of the agent-based model are conducted over a finite time interval in M miss distance cycles plus an additional collision cycle.

Next we explain key features of the IPS approach for two miss distance cycles and a collision cycle as illustrated in Figure 11. Here a circle represents a simulation object or particle, which describes the

simulation of the complete agent-based model (the total state space of all flights, systems, humans). In each cycle, N particles are simulated. In miss distance cycle c, the simulation of a particle is ended either if it has reached a simulation end time T (green circles in Figure 11), or if the miss distance boundary d_c has been reached by a pair of aircraft (circles ending on miss distance line in Figure 11). We denote the number of particles that have hit boundary d_c as N_c^h and the associated fraction of particles as $\gamma_c = N_c^h / N$. If no particle has hit d_c (i.e. if $N_c^h = 0$) then the MC IPS simulation stops at that boundary. The particles that have hit a miss distance boundary are stored.

At the start of each following cycle, resampling of the stored particles is used (illustrated by copying of particles in Figure 11). This means that the complete state space is copied, including all continuous states (e.g. aircraft positions and speeds, surveillance estimates, flight plans) and all discrete states (e.g. system modes, weather conditions, failure conditions). Next the simulation of each particle is continued independently, including its unique stochastic variations as defined in the agent-based model.

In the last simulation cycle (collision cycle), the simulation of each particle is proceeded until the simulation end time T. Here all collisions between aircraft pairs are counted as N^{coll}. Following a collision, the involved aircraft are removed from a particle, since the consequences of collision and potential ground impact are out of the scope of this study.

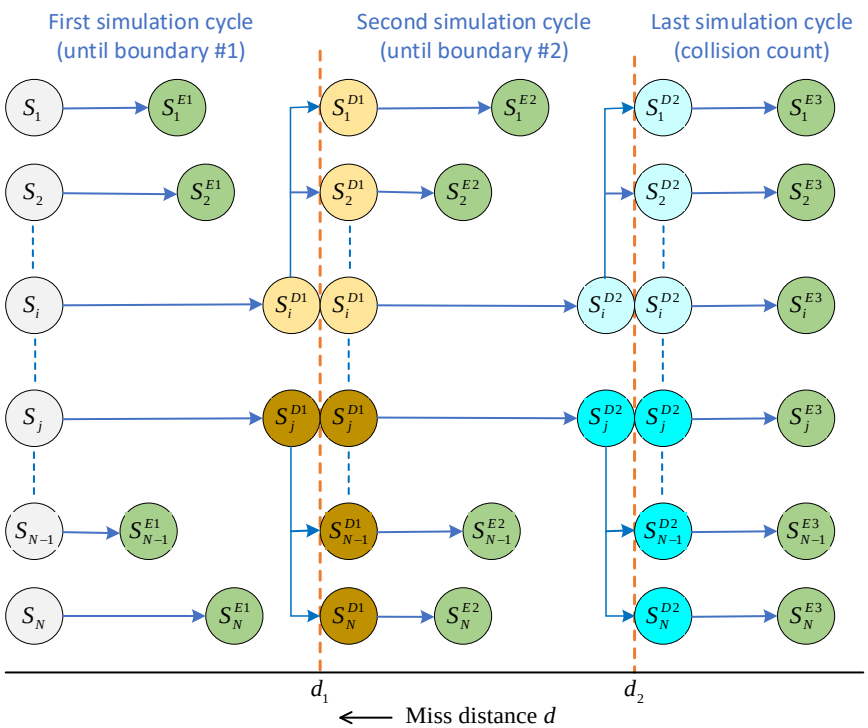

Figure 11. Illustration of IPS MC simulation for estimation of rare miss distances and collisions. Source: [54]

Based on the IPS MC simulation results, the probability that any aircraft pair reaches a miss distance boundary d_c in the time frame $(0,T)$ is estimated as

$$P(d_c) = \prod_{k=1}^{c} \gamma_k$$

and the expected number of aircraft pair collisions in the time frame $(0,T)$ is estimated as

$$m^{coll} = \frac{N^{coll}}{N} \prod_{k=1}^{M} \gamma_k \ .$$

By accounting for the number of flights and flight hours, above statistics can be extended to risk estimates per flight and per flight hour, as is customary in aviation. In this scheme setting $M = 0$ implies that no miss distance cycles are used, such that ordinary MC simulation for collision events remains.

5.3.2 Risk decomposition for global failure conditions

Risk decomposition for global failure conditions supports the assessment of rare global failures that would be encountered only very seldomly in IPS MC runs. For each global condition a simulation setting is defined as Working, Failing, or Stochastic.

- If the simulation mode is set as Working then the related system mode in the agent-based model is set as Working during a total simulation run. This is used to evaluate the conditional risk given that the mode is in the nominal condition.
- If the simulation mode is set as Failing then the related system mode in the agent-based model is switched from working to failing at a random time during a simulation run. Next, Bernoulli sampling is used to switch back from failing to working after a mean time of the global failure mode. This scheme is used to evaluate the conditional risk given that the mode is in the failure condition during a simulation run.
- If the simulation mode is set as Stochastic then the system mode in the agent-based model is updated at each time step in the simulations according to Bernoulli sampling. As such in this simulation mode no risk decomposition is used, but rather the system mode is switched between working and failing. This simulation mode can be used if the failure mode occurs sufficiently often.

In the simulations with risk decomposition, IPS MC simulation is performed for all possible combinations of working and failing global modes. For instance, a risk decomposition for two conditions leads to conditional risks for four cases (Working-Working, Working-Failing, Failing-Working, and Failing-Failing), which are multiplied with the probability of each case, and next the risk contributions of the four cases are summed to attain the overall risk.

6 Bibliography

[1] S. Stroeve, *Awareness in UTM operational concept*, D5.1, SymAware project, 2023.

[2] RTCA, *Minimum Operational Performance Standards (MOPS) for Detect and Avoid (DAA) Systems*, DO-365B, 2021.

[3] M. J. Kochenderfer, J. P. Chryssanthacopoulos, and R. E. Weibel, "A new approach for designing safer collision avoidance systems," in Ninth USA/Europe ATM R&D Seminar, Berlin, Germany, 2011.

[4] FAA, *Concept of operations for the Airborne Collision Avoidance System X*, Federal Aviation Administration, 2012.

[5] M. J. Kochenderfer, J. E. Holland, and J. P. Chryssanthacopoulos, "Next-generation airborne collision avoidance system," *Lincoln Laboratory Journal*, vol. 19, no. 1, pp. 17-33, 2012.

[6] M. J. Kochenderfer, *Decision making under uncertainty*, Cambridge (MA), USA: The MIT Press, 2015.

[7] R. S. Sutton, and A. G. Barto, *Reinforcement learning: An introduction*, Second edition ed., Cambridge (MA), USA: MIT Press, 2020.

[8] EUROCAE, *Minimum operational performance standards for Airborne Collision Avoidance System Xu (ACAS Xu): Volume I*, ED-275, 2020.

[9] EUROCAE, *Minimum operational performance standards for Airborne Collision Avoidance System Xu (ACAS Xu): Volume II Algorithm Design Description*, ED-275, 2020.

[10] L. E. Alvarez, I. Jessen, M. P. Owen, J. Silbermann, and P. Wood, "ACAS sXu: Robust Decentralized Detect and Avoid for Small Unmanned Aircraft Systems," in IEEE/AIAA 38th Digital Avionics Systems Conference (DASC), 2019, pp. 1-9. 10.1109/dasc43569.2019.9081631

[11] RTCA, *Minimum Operational Performance Standards for Airborne Collision Avoidance System sXu (ACAS sXu)*, RTCA DO-396, RTCA, Washington (DC), USA, 2022.

[12] A. Weinert, L. Alvarez, M. Owen, and B. Zintak, "Near Midair Collision Analog for Drones Based on Unmitigated Collision Risk," *Journal of Air Transportation*, vol. 30, no. 2, pp. 37-48, 2022/04/01, 2022. 10.2514/1.D0260

[13] C. Munoz, A. Narkawicz, G. Hagen, J. Upchurch, A. Dutle, M. Consiglio, and J. Chamberlain, "DAIDALUS: Detect and Avoid Alerting Logic for Unmanned Systems," in Proceedings of the 34th Digital Avionics Systems Conference (DASC 2015), Prague, Czech Republic, 2015.

[14] FAA, *Unmanned aircraft system (UAS) traffic management (UTM) concept of operations* Federal Aviation Administration, 2020.

[15] S. Stroeve, C. J. Villanueva-Cañizares, and G. Dean, "Remote pilot modelling for evaluation of ACAS Xu," in SESAR Innovation Days 2023, Sevilla, Spain, 2023.

[16] W.-C. Wang, M. G. Wu, and K. J. Monk, "Detect-and-Avoid Maneuver Planning: Benefits of Including Route Recapture," in AIAA Scitech 2021 Forum, 2021. 10.2514/6.2021-0451

[17] H. Erzberger, "Automated conflict resolution for air traffic control," 01/01, 2006.

[18] H. Erzberger, T. Lauderdale, and Y.-C. Chu, "Automated conflict resolution, arrival management and weather avoidance for ATM," 01/01, 2010.

[19] C. Cone, M. Owen, L. E. Alvarez, and M. W. Brittain, "Reward Function Optimization of a Deep Reinforcement Learning Collision Avoidance System," in AIAA SCITECH 2023 Forum, 2023. 10.2514/6.2023-2155

[20] M. Ribeiro, "Conflict Resolution at High Traffic Densities with Reinforcement Learning," Delft University of Technology, Delft, The Netherlands, 2023.

[21] M. Ribeiro, J. Ellerbroek, and J. Hoekstra, "Improving Algorithm Conflict Resolution Manoeuvres with Reinforcement Learning," *Aerospace,* vol. 9, no. 12, pp. 847, 2022. https://www.mdpi.com/2226-4310/9/12/847

[22] M. Ribeiro, J. Ellerbroek, and J. Hoekstra, "Distributed Conflict Resolution at High Traffic Densities with Reinforcement Learning," *Aerospace,* vol. 9, no. 9, pp. 472, 2022. https://www.mdpi.com/2226-4310/9/9/472

[23] M. Ribeiro, J. Ellerbroek, and J. Hoekstra, "Review of Conflict Resolution Methods for Manned and Unmanned Aviation," *Aerospace,* vol. 7, no. 6, pp. 79, 2020. https://www.mdpi.com/2226-4310/7/6/79

[24] M. Ribeiro, J. Ellerbroek, and J. Hoekstra, "Velocity Obstacle Based Conflict Avoidance in Urban Environment with Variable Speed Limit," *Aerospace,* vol. 8, no. 4, pp. 93, 2021. https://www.mdpi.com/2226-4310/8/4/93

[25] H. A. P. Blom, and G. J. Bakker, "Safety Evaluation of Advanced Self-Separation Under Very High En Route Traffic Demand," *Journal of Aerospace Information Systems,* vol. 12, no. 6, pp. 413-427, 2015. 10.2514/1.i010243

[26] G. J. Bakker, and H. A. P. Blom, *Report on Petri net modelling of the A3 operation,* iFly Deliverable D7.1c, 2010.

[27] H. A. P. Blom, and G. J. Bakker, *Final Report on Accident Risk Assessment of the A3 operation,* iFly Deliverable D7.4, 2011.

[28] P. Fiorini, and Z. Shiller, "Motion Planning in Dynamic Environments Using Velocity Obstacles," *The International Journal of Robotics Research,* vol. 17, no. 7, pp. 760-772, 1998. 10.1177/027836499801700706

[29] G. A. M. Velasco, C. Borst, J. Ellerbroek, M. M. v. Paassen, and M. Mulder, "The Use of Intent Information in Conflict Detection and Resolution Models Based on Dynamic Velocity Obstacles," *IEEE Transactions on Intelligent Transportation Systems,* vol. 16, no. 4, pp. 2297-2302, 2015. 10.1109/TITS.2014.2376031

[30] P. Zhao, and Y. Liu, "Physics Informed Deep Reinforcement Learning for Aircraft Conflict Resolution," *IEEE Transactions on Intelligent Transportation Systems,* vol. PP, pp. 1-14, 05/13, 2021. 10.1109/TITS.2021.3077572

[31] H. Y. Ong, and M. J. Kochenderfer, "Markov Decision Process-Based Distributed Conflict Resolution for Drone Air Traffic Management," *Journal of Guidance, Control, and Dynamics,* vol. 40, no. 1, pp. 69-80, 2017. 10.2514/1.G001822

[32] J. Yang, D. Yin, Y. Niu, and L. Shen, "Distributed Cooperative Onboard Planning for the Conflict Resolution of Unmanned Aerial Vehicles," *Journal of Guidance, Control, and Dynamics,* vol. 42, no. 2, pp. 272-283, 2019. 10.2514/1.G003583

[33] W. Zhao, N. Wang, J. Yang, J. Wang, and S. Chen, "Online cooperative airspace conflict resolution of unmanned aerial vehicles by space mapping based iterative search method," *Advances in Mechanical Engineering,* vol. 14, pp. 168781322211238, 09/12, 2022. 10.1177/16878132221123886

[34] P. Zhao, H. Erzberger, and Y. Liu, "Multiple-Aircraft-Conflict Resolution Under Uncertainties," *Journal of Guidance, Control, and Dynamics,* vol. 44, pp. 1-19, 09/06, 2021. 10.2514/1.G005825

[35] P. Zhao, W. Wang, L. Ying, B. Sridhar, and Y. Liu, "Online Multiple-Aircraft Collision Avoidance Method," *Journal of Guidance, Control, and Dynamics*, vol. 43, no. 8, pp. 1456-1472, 2020. 10.2514/1.G005161

[36] K. Karur, N. Sharma, C. Dharmatti, and J. E. Siegel, "A Survey of Path Planning Algorithms for Mobile Robots," *Vehicles*, vol. 3, no. 3, pp. 448-468, 2021. https://www.mdpi.com/2624-8921/3/3/27

[37] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments," *The International Journal of Robotics Research*, vol. 29, no. 5, pp. 485-501, 2010. 10.1177/0278364909359210

[38] G. Gugan, and A. Haque, "Path Planning for Autonomous Drones: Challenges and Future Directions," *Drones*, vol. 7, no. 3, pp. 169, 2023. https://www.mdpi.com/2504-446X/7/3/169

[39] L. De Filippis, G. Guglieri, and F. Quagliotti, "Path Planning Strategies for UAVS in 3D Environments," *Journal of Intelligent & Robotic Systems*, vol. 65, no. 1, pp. 247-264, 2012/01/01, 2012. 10.1007/s10846-011-9568-2

[40] Y. Wu, and K. H. Low, "An Adaptive Path Replanning Method for Coordinated Operations of Drone in Dynamic Urban Environments," *IEEE Systems Journal*, vol. 15, no. 3, pp. 4600-4611, 2021. 10.1109/JSYST.2020.3017677

[41] Y. Lin, and S. Saripalli, "Sampling-Based Path Planning for UAV Collision Avoidance," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 11, pp. 3179-3192, 2017. 10.1109/TITS.2017.2673778

[42] EUROCAE, *Minimum Operational Performance Standards for Airborne Collision Avoidance System X (ACAS X) (ACAS Xa AND ACAS Xo)*, ED-256, Saint-Denis, France, 2018.

[43] EUROCAE, *Minimum Operational Performance Standards for Airborne Collision Avoidance System X (ACAS X) (ACAS Xa AND ACAS Xo)*, ED-256A, Saint-Denis, France, 2023.

[44] S. Stroeve, "What Matters in the Effectiveness of Airborne Collision Avoidance Systems? Monte Carlo Simulation of Uncertainties for TCAS II and ACAS Xa," *Aerospace*, vol. 10, no. 11, pp. 952, 2023. https://www.mdpi.com/2226-4310/10/11/952

[45] J. E. Holland, M. J. Kochenderfer, and W. A. Olson, "Optimizing the next generation collision avoidance system for safe, suitable, and acceptable operational performance," in Tenth USA/Europe Air Traffic Management Research and Development Seminar (ATM2013), Chicago, USA, 2013.

[46] M. P. Owen, A. Panken, R. Moss, L. Alvarez, and C. Leeper, "ACAS Xu: Integrated collision avoidance and detect and avoid capability for UAS," in 2019 IEEE/AIAA 38th Digital Avionics Systems Conference (DASC), 2019, pp. 1-10.

[47] D. P. Bertsekas, "Rollout algorithms for discrete optimization: A survey," *Handbook of combinatorial optimization*, pp. 2989-3013: Springer New York, 2013.

[48] S. Stroeve, C. J. Villanueva Cañizares, and G. Dean, "The critical impact of remote pilot modelling in evaluation of detect-and-avoid systems explained for ACAS Xu," Zenodo, ed., 2024. https://doi.org/10.5281/zenodo.10890481

[49] S. Stroeve, C. J. Villanueva Cañizares, and G. Dean, "Remote pilot modelling for evaluation of ACAS Xu," in SESAR Innovation Days 2023, Seville, Spain, 2023. https://doi.org/10.61009/SID.2023.1.14

[50] E. Klintberg, J. Dahl, J. Fredriksson, and S. Gros, "An improved dual Newton strategy for scenario-tree MPC," 2016. 10.1109/CDC.2016.7798822

[51] L. E. Dubins, "On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents," *American Journal of mathematics*, vol. 79, no. 3, pp. 497-516, 1957.

[52] S. Russell, and P. Norvig, *Artificial Intelligence: A Modern Approach*, Fourth edition ed.: Pearson, 2021.

[53] J. E. Hopcroft, J. T. Schwartz, and M. Sharir, “On the Complexity of Motion Planning for Multiple Independent Objects; PSPACE- Hardness of the "Warehouseman's Problem",” *The International Journal of Robotics Research,* vol. 3, no. 4, pp. 76-88, 1984. 10.1177/027836498400300405

[54] S. Stroeve, B. Bakker, C. J. Villanueva Cañizares, and N. Fota, “Close proximity and collision risk assessment of drones and urban air mobility,” in 11th SESAR Innovation Days, Virtual conference, 2021.

[55] H. A. P. Blom, J. Krystul, G. J. Bakker, M. B. Klompstra, and B. Klein Obbink, "Free flight collision risk estimation by sequential Monte Carlo simulation," *Stochastic hybrid systems; recent developments and research trends,* C. G. Cassandras and J. Lygeros, eds., pp. 249-281, Boca Raton, USA: CRC Press, 2007.

[56] H. A. P. Blom, H. Ma, and G. J. Bakker, “Interacting Particle System-based Estimation of Reach Probability for a Generalized Stochastic Hybrid System,” *IFAC-PapersOnLine,* vol. 51, no. 16, pp. 79-84, 2018/01/01/, 2018. 10.1016/j.ifacol.2018.08.014

[57] F. Cerou, P. Del Moral, and F. Le Gland, “Limit theorems for the multilevel splitting algorithms in the simulation of rare events,” in Proceedings of Winter Simulation Conference, Orlando (FL), USA, 2005.